



Format-Aware Fusion for Fast FP4 Pretraining

Native MXFP4 and NVFP4 Training on FP4-Compatible Hardware

REPORT FOCUS

Native FP4 training, format-aware fusion, and measured speed–quality trade-offs for the Llama 3 model family.

Robert Hu

Correspondence: robert.stats.hu@gmail.com

September 2026

© 2026 Robert Hu. Original manuscript and figures licensed under [CC BY 4.0](#).

Abstract

Four-bit floating-point (FP4) Tensor Cores accelerate matrix multiplication, but scale computation, operand packing, layout construction, and saved backward state can erase the gain. We present *format-aware fusion*, which co-designs each quantization producer with its scale domain and consumer layout for native MXFP4, global NVFP4, and cooperative-thread-array-local NVFP4. We evaluate Llama-3-family 8B pretraining through 160 billion tokens using bfloat16 output projections and compiled cross entropy. In matched same-accelerator probes, bfloat16 and Transformer Engine NVFP4 reach 18.8K and 27.6K tokens/s/GPU, while our fastest custom route reaches 37.9K. MXFP4 with row-gradient stochastic rounding and fixed-sign 32-value Hadamard weight-gradient preconditioning reaches 37.2K tokens/s/GPU (86.3% bfloat16 model FLOP utilization) and ends 2.11% above the raw bfloat16 training-loss endpoint. A Transformer Engine recipe with four final bfloat16 blocks ends 0.87% above bfloat16 at 27.1K tokens/s/GPU. Downstream rankings differ from training-loss rankings, showing that FP4 outcomes depend jointly on scale contract, operand, and execution path.

1 Introduction

Four-bit floating-point (FP4) Tensor Cores accelerate general matrix multiplication (GEMM), but each training layer must also produce scales, pack values, build forward and backward layouts, and retain autograd state. When separate kernels move bfloat16 (BF16) intermediates through high-bandwidth memory (HBM), this surrounding work can erase the GEMM speedup [32].

We address this gap with *format-aware fusion*: each producer completes the scale shared by its output values while emitting the packed layouts and saved state needed by the following matrix multiplications. We use *route* to mean a complete numerical recipe and execution path, not only an FP4 datatype. MXFP4 completes a local 32-value scale, global NVFP4 waits for a tensor-wide maximum, and cooperative-thread-array-local (CTA-local) NVFP4 completes two outer scales inside the tile that owns the matrix multiplication. The profitable fusion boundary therefore differs by format.

We evaluate the resulting paths in Llama-3-family 8B pretraining at sequence length 8,192 and global batch 512. Same-accelerator probes measure every route with a BF16 output projection and compiled cross entropy (CE), while long trajectories cover up to 160B tokens. The fastest custom route reaches 37.9K tokens/s/GPU. The MXFP4 route with row-gradient stochastic rounding (SR) and fixed-sign 32-value Hadamard (H32) weight-gradient (Wgrad) preconditioning reaches 37.2K tokens/s/GPU and ends 2.11% above the raw BF16 endpoint. The TE-native and four-final-BF16-block trajectories end 1.34% and 0.87% above raw BF16, respectively, at measured throughputs of 27.6K and 27.1K tokens/s/GPU.

Table 1 situates the implementation relative to established FP4 practice. We contribute:

- *Numerical parameterization.* We specify the FP4 contract per operand: upward-rounded MXFP4 scales for activations and gradients; an adaptive finer scale for 32×32 weight tiles; separate FP32 row- and column-tile scale factors for CTA-local NVFP4; stochastic rounding only for the data-gradient multiplication; and paired fixed-sign 16- or 32-value Hadamard preconditioning only for the weight-gradient multiplication.
- *Format-aware fusion and native implementation.* We implement custom MXFP4, custom global NVFP4, and CTA-local NVFP4 producers that finish the scale calculation over the required group of values and emit the layouts and saved state required by forward and backward.

The MXFP4 and CTA-local weight producers quantize once and emit both packed weight layouts in the same fused operation. Custom global NVFP4 likewise integrates its shared 16×16 weight encoding and both packed layouts directly into the fused path. Backward stages fuse optional Hadamard preconditioning with column quantization. The same framework supports depth- and operand-wise hybrids.

- *End-to-end empirical evaluation.* We implement and evaluate the custom pure and hybrid routes alongside BF16 and TE comparators. The hybrid designs include depth-wise format assignment and an operand-wise assignment motivated by a data-gradient error analysis using identical saved tensors. The evidence comprises matched throughput probes, trajectories through 160B training tokens, checkpoint-matched validation loss on a fixed external data stream, and downstream evaluation of the complete recipes. We release the training and evaluation code, pinned kernel sources, complete recipe ledger, publication data, and figure builders at <https://github.com/MrHuff/mfu-fp4-training>; Appendix C.6 describes its reproducibility contract.

2 FP4 Contracts and Numerical Method

2.1 Related work

Low-precision training builds on mixed-precision accumulation and stochastic rounding [21, 27]. Early 4-bit systems combined integer weights and activations with specialized gradient formats [40]. Chmiel et al.’s logarithmic unbiased quantization (LUQ) targets unbiased neural-gradient quantization over the full range: it combines stochastic underflow below the smallest nonzero value, range selection that avoids upper clipping, and stochastic rounding within a logarithmic FP4 grid [7]. This is a stronger construction than applying ordinary adjacent-bin SR only inside the representable range. FP8 and microscaling then established floating-point formats with shared block exponents [28, 33, 37, 38]. Their behavior depends on more than bit width: scale format, block geometry, tensor statistics, and the contraction being quantized all matter [17, 24, 41, 47].

Recent FP4 pretraining systems make different choices along those axes. Tseng et al. apply MXFP4 to the two backward matrix multiplications and combine stochastic rounding with randomized Hadamard transform (RHT) preconditioning [44]. FP4 All the Way uses round-to-nearest in Fprop and SR in the backward and update contractions; in Wgrad, it stochastically rounds both the neural-gradient and activation operands [8]. NVIDIA’s NVFP4 recipe combines two-dimensional weights, gradient stochastic rounding, paired Wgrad preconditioning, and selected higher-precision linears [32]. Quartet and FP4 All the Way study fully low-precision paths, while subsequent work develops native full-pipeline MXFP4 and unbiased NVFP4 gradient estimation [4, 8, 9, 35]. Other FP4 studies examine optimizer and oscillation effects, adaptive block scales, outlier dynamics, shrinkage bias, and quantization beyond the linear operators [1, 5, 6, 10, 13, 14, 45, 49, 50]. Complementary systems explore spectral-domain FP4, alternative four-bit formats, and native FP4 training for mixture-of-experts models [3, 15, 42, 48]. UE5M3 FP4 reallocates the unused sign bit of an E4M3 block scale to a fifth exponent bit. Hu et al. combine this wider unsigned scale with periodic sample-and-hold tensor scaling and selective backward SR, while omitting RHT and using FP4 in every eligible internal linear [25].

We abbreviate the forward, input-gradient, and weight-gradient matrix multiplications as Fprop, Dgrad, and Wgrad, respectively.

Three mechanisms directly shape this work. A weight encoding can remain numerically consistent under transposition [9, 32, 36]. Stochastic rounding (SR) can remove within-bin rounding bias, while its placement and treatment of underflow and saturation—clipping to the largest representable magnitude—remain part of the quantizer contract [7, 21, 34]. Paired Hadamard preconditioning spreads outliers while preserving the unquantized dot product [2, 9, 32, 44]. We specialize where these mechanisms act in each contraction and realize the complete operand contract inside fused training kernels.

The systems contribution is to optimize the quantizer, its physical layouts, the consuming matrix multiplication, and retained backward state as one execution boundary. This follows the broader principle that data movement and producer–consumer structure determine realized model speed [11, 12, 20, 23, 39, 46].

E2M1 denotes the four-bit value format with one sign bit, two exponent bits, and one fraction bit. E8M0 is a power-of-two scale format, E4M3 is an eight-bit floating-point scale format, and *amax* denotes the maximum absolute value. A 2D16 or 2D32 weight encoding shares one numerical scale over a 16×16 or 32×32 tile and reuses it when the weight is viewed in either orientation. H16 and H32 denote Hadamard transforms that mix 16 or 32 values. Table 1 pairs each existing method with its implementation in this work; the following subsections explain these choices in detail.

2.2 The same four-bit values, three different scale contracts

All three custom routes ultimately present E2M1 operands—one sign bit, two exponent bits, and one fraction bit—to a Blackwell Tensor Core. Their numerical and scheduling behavior nevertheless differs because they assign scales to different domains. A useful common notation for one scalar is

$$\hat{z}_i = S \delta_{b(i)} Q_{\text{E2M1}} \left(\frac{z_i}{S \delta_{b(i)}} \right), \quad (1)$$

where $\delta_{b(i)}$ is the effective block dequantization multiplier, S is an optional outer scale, and Q_{E2M1} rounds and saturates to E2M1. The scale choice determines both quantization error and when the producer may safely release its output.

Table 1: Existing numerical and systems methods and their parameterization, composition, or implementation in this work.

Component	Existing method	Implementation in this work
MXFP4 scale policy	E8M0 forces the block scale to a power of two. Rounding the exponent up keeps the block maximum within the largest finite E2M1 magnitude, 6; rounding it down halves the decoded grid spacing while clamping the upper tail to that endpoint [9, 33]	Scale direction is explicit and operand-specific. Activations and gradients use upward, encode-centric scales; 2D32 weights admit the next finer scale only under the fixed tile-maximum test. Fused producers implement both rules
CTA-local outer scaling	NVFP4 uses one tensor-level FP32 outer scale together with an E4M3 scale for each 16 values [32, 33]	We factor the outer correction into FP32 row- and column-tile factors that remain fixed along K. Their product is applied after the full Tensor Core accumulation, replacing the tensor-wide maximum-magnitude reduction with tile-local scale completion
2D weight contract	NVFP4 uses orientation-consistent 2D16 weights, and existing MXFP4 systems describe transpose-consistent 2D32 weights [9, 31, 32, 36]	Pure-format and depth-wise routes preserve one weight encoding across Fprop and Dgrad. Custom global NVFP4 returns both packed views under one 2D16 encoding; MXFP4 and CTA-local fused producers quantize once and emit both layouts. The operand hybrid instead emits distinct encodings from the same BF16 tile
Stochastic-rounding placement	Chmiel et al. use adjacent-bin stochastic rounding in Dgrad and on both Wgrad operands; NVIDIA applies gradient SR to both Dgrad and Wgrad [8, 32]	One producer creates two consumer-specific dY representations: a stochastically rounded row-facing operand for Dgrad and a deterministic column-facing representation for Wgrad
Wgrad preconditioning	Existing FP4 recipes use paired randomized or deterministic Hadamard transforms [9, 32, 44]	In signed-Hadamard routes, fixed-sign H16/H32 reuses the fixed 16-entry sign pattern from the pinned TE implementation; the plain-H16 hybrid uses no sign flips. Each acts only on paired X, dY Wgrad representations and is fused with column quantization. Dgrad and the stored weight are unchanged
Format-aware fusion	IO-aware fusion and the underlying MXFP4/NVFP4 scale formats are documented in earlier systems [12, 20, 32, 33]	The fusion boundary is set by when the required scale becomes available. Query/key/value, rotary-position, and gated feed-forward stages produce packed layouts and saved backward tensors together; custom global NVFP4 overlaps tensor amax while CTA-local removes that dependency
Operand-wise hybrid	Existing recipes vary precision by layer or module [32, 36, 50]	We assign MXFP4 or CTA-local NVFP4 per contraction inside every linear and produce both representations together from shared BF16 inputs, motivated by comparing Dgrad error on identical saved tensors

Table 2: The FP4 execution contracts used in this report. CTA-local NVFP4 changes the outer-scale domain while retaining E2M1 values and E4M3 block scales.

Route	Block	Scale metadata	Main systems consequence
MXFP4	32	E8M0 block scale	Local reduction; coarse power-of-two grid
Custom global NVFP4	16	E4M3 block + FP32 tensor	Requires the completed tensor amax
CTA-local NVFP4	16	E4M3 block + two FP32 tile-outer scales	Tile-local production; two-sided metadata

2.2.1 MXFP4 encode- and decode-centric scale selection

E8M0 represents a power-of-two scale with eight exponent bits and no fraction; E4M3 is an FP8 scale with four exponent and three fraction bits. In MXFP4, $S = 1$, s_b is an E8M0 power of two shared by 32 values, and the effective multiplier in Equation 1 is $\delta_b = s_b/6$. The block maximum therefore produces a scale locally, but adjacent scale levels differ by exactly two. This leaves a consequential choice about which neighboring power of two to store.

We call upward exponent rounding *encode-centric* scaling because it first encloses every source value in a scale that E2M1 can encode without upper-tail clipping. For a positive, finite block maximum $a_b = \max_i |z_i|$ within the supported E8M0 exponent range, it computes

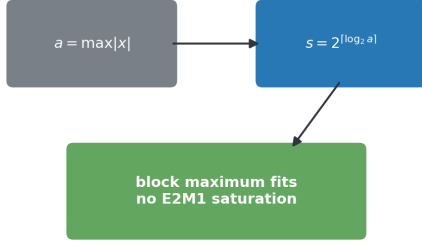
$$s_b = 2^{\lceil \log_2 a_b \rceil}, \quad q_i = Q_{\text{E2M1}}\left(\frac{6z_i}{s_b}\right), \quad \hat{z}_i = \frac{s_b}{6}q_i. \quad (2)$$

The largest magnitude then maps to at most the finite E2M1 endpoint, 6. We call downward exponent rounding *decode-centric* scaling: it chooses $s_b = 2^{\lfloor \log_2 a_b \rfloor}$ so the decoded multiplier $s_b/6$ is half as large whenever a_b is not already a power of two. That halves the grid spacing for values that remain in range, but values above the endpoint saturate to 6 and lose their excess magnitude. In plain terms, encode-centric scaling protects the largest value; decode-centric scaling represents smaller values more finely by accepting clipping at the top of the block. For example, if $a_b = 10$, encode-centric scaling stores $s_b = 16$ and does not clip; decode-centric scaling stores $s_b = 8$, halves the spacing, and clips values whose magnitude exceeds 8.

We expose this direction as part of the MXFP4 route rather than treating it as an incidental implementation detail. Development-only MXFP4 pretraining screens favored encode-centric over decode-centric scaling for both activation and gradient operands. These screens selected the recipe; they are not a reported quantitative comparison. Accordingly, every reported MXFP4 experiment rounds activation and gradient block scales upward, as shown in Equation 2. This choice protects the largest value in each block from clipping. The weight rule described next is the only place where our MXFP4 recipe sometimes selects the finer downward scale.

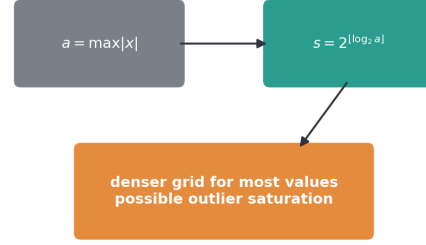
Weights use a different trade-off. The orientation-consistent 2D32 producer starts from the safe upward-rounded scale s_T for the maximum a_T over a 32×32 tile. For a nonzero tile with a representable safe exponent, it selects $s_T/2$ only when $a_T/s_T < 0.87$. The smaller scale halves the quantization spacing and accepts a fixed, bounded amount of upper-tail clipping. The same fused producer makes that decision once and emits both physical consumer layouts. Our MXFP4 implementation makes the operand-dependent scale policy explicit and performs the scale decision

Encode-centric scale: round amax upward



Encode $Q(6x/s)$ and reconstruct $(s/6)Q$; the maximum stays finite.

Decode-centric scale: round amax downward



The same decode rule uses a grid up to 2x denser, but large values may clip.

Figure 1: Operand-specific E8M0 scale selection. Encode-centric scaling rounds the block maximum upward and keeps every input within the E2M1 endpoint. For a non-power-of-two maximum, decode-centric scaling rounds downward, halves the decoded grid spacing, and clips the block tail. Our route applies this range–resolution choice per operand: activations and gradients use the encode-centric rule, while 2D32 weights select the finer scale only under the fixed $a_T/s_T < 0.87$ condition.

and dual-layout production inside the same producer. Figure 1 summarizes the resulting operand-dependent range–resolution trade-off.

2.2.2 Global and CTA-local NVFP4 outer scales

Global NVFP4 instead uses an E4M3 scale for each 16 values and an FP32 tensor scale S [32, 33]. Its local grid is finer, but every block depends on the completed tensor amax, the maximum absolute value. Our *custom global NVFP4* path preserves the tensor-global numerical contract used by TE-native NVFP4 while replacing generic quantize–layout–GEMM boundaries with custom producers and consumers.

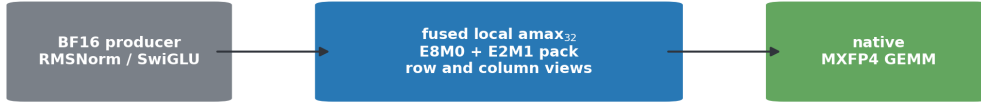
This tensor-wide maximum also creates a scheduling dependency. A GPU divides the tensor among many cooperative thread arrays (CTAs), which are thread blocks that may run at different times. Each CTA can find a partial maximum, but the final tensor maximum is unavailable until all partial results have been combined. Packing the quantized tensor and starting the dependent matrix multiplication must wait for that result. The wait forms a tensor-wide synchronization boundary within the GPU and can leave a latency bubble even when individual tiles are otherwise ready.

The CTA-local route removes that cross-CTA dependency by changing only the outer-scale domain. Each CTA computes maxima for the row and column tiles it already owns, derives their outer factors using synchronization within that CTA, and can then proceed to its matrix-multiplication tile without waiting for unrelated CTAs. It does not remove synchronization altogether; it confines scale completion to one CTA. For one matrix-multiplication output tile, an A-row tile owns factor α_i and a B-column tile owns factor β_j . Both remain fixed over the complete K reduction. If $\tilde{A}$ and $\tilde{B}$ denote E2M1 values decoded with their E4M3 per-block scales, often called microscale, then

$$C_{ij} = \alpha_i \beta_j \sum_k \tilde{A}_{ik} \tilde{B}_{kj}. \quad (3)$$

The scale contract determines the fusion and synchronization boundary

MXFP4



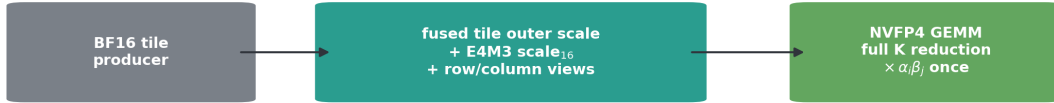
Power-of-two scales are local to 32 values: quantization can finish as tiles are produced.

Custom global NVFP4



The finer scale grid adds a tensor-wide completion dependency; overlap matters as much as packing speed.

CTA-local NVFP4



No tensor-wide amax barrier: two FP32 outer scales stay constant along the inner dimension and are applied at GEMM output.

Figure 2: Scale-completion boundaries for the three custom routes. MXFP4 finalizes a local 32-value scale; global NVFP4 waits for tensor amax; CTA-local NVFP4 computes tile-owned row and column outer factors without waiting for a tensor-wide reduction. The completion point determines which preparation work can be fused with its matrix multiplication.

The Tensor Core accumulates the inner sum and the epilogue applies $\alpha_i\beta_j$ once. This factorization is valid only because the two outer scales are invariant in K , the contraction dimension. Keeping both corrections in FP32 metadata also avoids quantizing those corrections onto the coarser FP8 scale grid. Figures 2 and 3 show, respectively, where each format completes its scale state and how the CTA-local factors replace the tensor-wide outer-scale dependency.

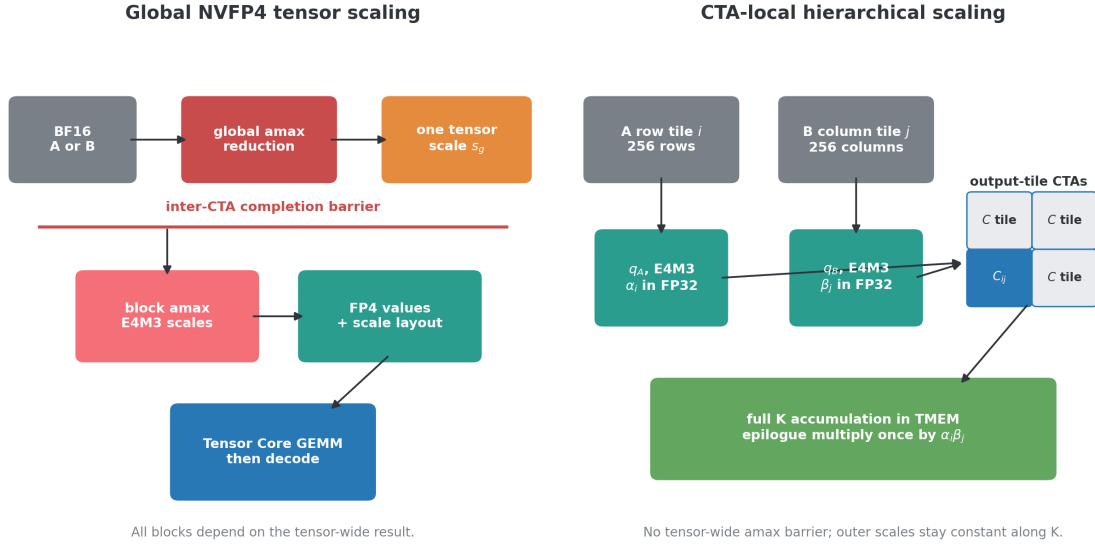


Figure 3: Tensor-global and CTA-local NVFP4 outer-scale domains. Global NVFP4 gates block packing on tensor amax. Our CTA-local route carries tile-owned row and column outer factors through the full K accumulation and applies $\alpha_i \beta_j$ once in the epilogue.

2.3 Why forward, data gradient, and weight gradient need different views

For a linear layer $Y = XW^T$, forward propagation (Fprop), the input or data gradient (Dgrad), and the weight gradient (Wgrad) consume different views of the same tensors. Let Q_r denote a row-facing representation and Q_c a column-facing representation. Both preserve the logical tensor shape; the subscript identifies the physical layout and scale orientation used by the consumer. Suppressing decode and scale metadata, training evaluates three contractions:

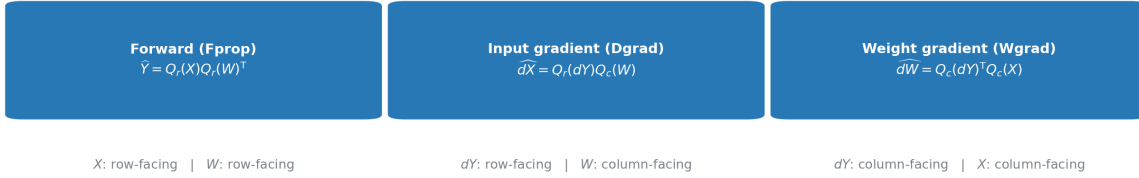
$$\hat{Y} = Q_r(X)Q_r(W)^T, \quad \text{Fprop,} \quad (4)$$

$$\widehat{dX} = Q_r(dY)Q_c(W), \quad \text{Dgrad,} \quad (5)$$

$$\widehat{dW} = Q_c(dY)^T Q_c(X), \quad \text{Wgrad.} \quad (6)$$

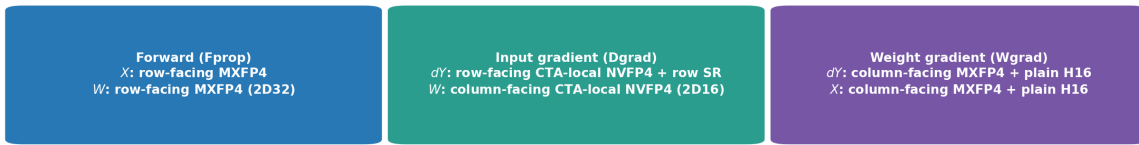
Thus one saved gradient needs a row-oriented representation for Dgrad and a column-oriented representation for Wgrad; weights likewise participate in two orientations. “Quantize the tensor to FP4” is incomplete unless it also states which contraction consumes it. The upper panel of Figure 4 maps these row- and column-facing representations to the three contractions.

A linear layer needs a different operand view for each contraction



r and c denote consumer-facing row- and column-packed scale/layout views; the logical shape does not change.

Operand hybrid: numerical format and view for every matrix input



One fused backward producer turns a BF16 dY tile into both required FP4 views.

Fixed H32 changes only the paired Wgrad transform; the formats and row/column views stay the same.

Figure 4: Formats and packed views for the plain-H16 operand hybrid. Here r and c denote the consumer-facing row- and column-packed scale/layout representations; they do not change a tensor’s logical shape. Top: Fprop consumes row-facing X and W , Dgrad consumes row-facing dY and column-facing W , and Wgrad consumes column-facing dY and X . Bottom: Fprop uses MXFP4 X_r and 2D32 W_r ; Dgrad uses CTA-local NVFP4 dY_r with row stochastic rounding (SR) and 2D16 W_c ; Wgrad uses MXFP4 dY_c and X_c with paired plain-H16 preconditioning. One fused backward producer emits both dY representations from the BF16 tile. Wgrad does not read W because it produces dW . Plain H16 has no sign flips; fixed H32 changes only the paired Wgrad preconditioner.

2.4 Two-dimensional weights: numerical consistency versus implementation

Independent one-dimensional weight quantizers can produce $W_{\text{fwd}} = Q_K(W)$ for Equation 4 and a different $W_{\text{bwd}} = Q_N(W)$ for Equation 5. Forward and backward then consume different numerical approximations of the same BF16 weight. Two-dimensional weight quantization instead assigns one scale to each $b \times b$ weight tile. Let q_{max} be the largest finite E2M1 magnitude and let $\text{scales}_{\mathcal{S}}$ denote the route’s scale-selection rule on its representable scale set $\mathcal{S}$. Then

$$a_{pq} = \max_{i \in p, j \in q} |W_{ij}|, \quad s_{pq} = \text{scales}_{\mathcal{S}}\left(\frac{a_{pq}}{q_{\text{max}}}\right), \quad \widehat{W}_{ij} = S_{s_{pq}} Q_{\text{E2M1}}\left(\frac{W_{ij}}{S_{s_{pq}}}\right), \quad (7)$$

and reuses that numerical encoding in both orientations. The row and column layouts can differ in memory, but their decoded weight values agree. This is the consistency motivation identified by NVIDIA and by work on transposition-invariant block quantization [32, 36]. Equivalently, the numerical invariant is

$$Q_r^{2D}(W) \equiv Q_c^{2D}(W) \equiv \widetilde{W}, \quad (8)$$

even though the two memory layouts contain differently ordered bytes. Figure 5 contrasts independent one-dimensional quantization with this shared numerical encoding.

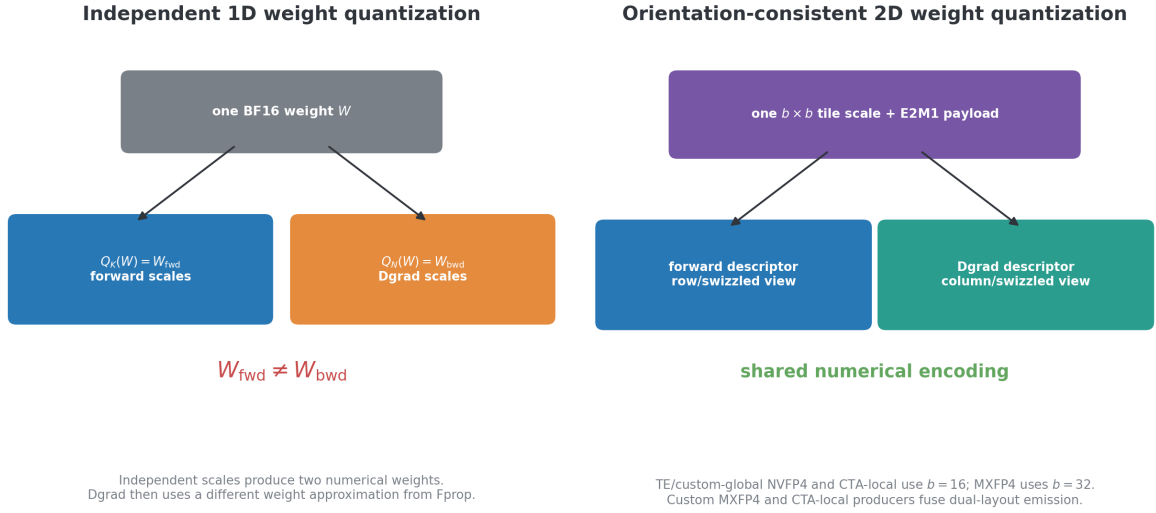


Figure 5: Independent one-dimensional scales can make Fprop and Dgrad consume different numerical weights. Orientation-consistent 2D quantization computes one tile encoding and exposes it through row- and column-oriented memory layouts. Our implementation optimization fuses production of those layouts into the custom MXFP4 and CTA-local producers under their respective scale policies.

Transformer Engine (TE), custom global NVFP4, and the CTA-local path all preserve one numerical weight encoding across the two 16×16 consumer orientations. In TE’s 2D16 contract, one scale belongs to each 16×16 weight tile and is replicated into the 1×16 scale layouts required by the Tensor Core consumers [31, 32]. “2D” therefore describes shared numerical scale ownership across orientations; the matrix-multiplication shape is unchanged. TE and custom global NVFP4 use the tensor-level NVFP4 outer scale; the custom route emits the packed values, block scales, and both physical weight layouts directly. CTA-local NVFP4 retains the 2D16 consistency invariant. Its weight producer finds the local weight maxima and E4M3 block scales, folds the relative corrections into those scales, and publishes one common FP32 weight outer factor for both orientations. Activations and gradients continue to use the tile-owned row and column factors in Equation 3. Our MXFP4 path applies the same orientation-consistency objective to 32×32 MX tiles. Tseng et al.’s backward-only MXFP4 method instead quantizes each operand along the reduction axis of the current contraction [44]. Transpose-consistent 2D32 MXFP4 is also described in existing systems [9, 36].

The numerical 2D contract follows the cited 2D16 and 2D32 methods. Our implementation optimization is producer fusion: the MXFP4 and CTA-local producers each load the BF16 weight tile once, complete the route’s scale reduction, quantize one numerical encoding, and emit the row- and column-oriented physical layouts within the same producer boundary. The MXFP4 producer incorporates the adaptive 2D32 scale rule in Section 2.2.1; the CTA-local producer combines its 2D16 encoding with local maximum and scale calculation, then normalizes both layouts to a common weight outer factor. This construction removes a second independent scale decision and guarantees that Fprop and Dgrad decode the same numerical weight even though their physical layouts differ.

Equation 8 applies to the pure routes and the depth-wise hybrid. The operand-wise hybrid is an explicit exception: it reads the BF16 weight once, then emits a 2D32 MX weight encoding for Fprop and a 2D16 CTA-local weight encoding for Dgrad. Wgrad does not consume W ; it separately

quantizes X and dY with MXFP4. This gives each contraction a different error profile, so end-to-end training must evaluate the tradeoff.

2.5 Stochastic rounding and signed Hadamard preconditioning

For adjacent representable values q_- and q_+ , data stochastic rounding (SR) uses

$$\Pr[Q_{\text{SR}}(z) = q_+] = \frac{z - q_-}{q_+ - q_-}, \quad \Pr[Q_{\text{SR}}(z) = q_-] = 1 - \Pr[Q_{\text{SR}}(z) = q_+], \quad (9)$$

which is unbiased between the two grid points before saturation [21, 34]. This is the ideal probability rule; implementations that use too few random bits can introduce bias [18]. Equation 9 is ordinary adjacent-bin SR, not LUQ. LUQ also stochastically handles values below the logarithmic FP minimum and selects the range to avoid clipping [7]. Our microscaled E2M1 paths can saturate, so we claim removal of deterministic bias between adjacent finite values, not unbiasedness of the complete saturated quantizer.

The placement also differs. FP4 All the Way applies SR to the neural gradient entering Dgrad and to both the neural-gradient and activation operands entering Wgrad [8]. NVIDIA recommends round-to-nearest for weights and activations and SR for the gradient input to both Dgrad and Wgrad [32]. Our custom long routes are narrower: data SR is enabled only on the row-oriented dY representation consumed by Dgrad. The Wgrad column representation of dY , weights, activations, and all scale values remain deterministic. This axis-selective placement defines the custom backward operand contract. The row/column separation is visible in Figure 4; Figure 6 locates the stochastically rounded Dgrad input and paired transformed Wgrad inputs in the fused backward schedule.

Hadamard preconditioning addresses a different failure mode. One outlier can set the scale for a whole block, leaving fewer useful E2M1 values for the other entries. A Hadamard transform mixes the block with additions and subtractions before quantization, spreading a large coordinate over the block. In Wgrad, this can keep smaller contributions from underflowing to zero. The transform must be applied to both inputs of the weight-gradient dot product so that it cancels in exact arithmetic [2, 32, 44]. If H is the normalized block Hadamard matrix and $R = HD$ uses a sign diagonal D , then

$$dW = dY^T X = (RdY)^T (RX), \quad R^T R = I. \quad (10)$$

Both Wgrad operands must therefore be transformed as a pair. Transforming only dY changes the gradient rather than only changing its quantization basis.

The Hadamard matrix itself is deterministic. In an RHT, randomness comes from the diagonal sign vector D , and that vector need not be resampled at every step. Tseng et al. sample signs and transform both MXFP4 backward contractions, (dY, W) for Dgrad and (dY, X) for Wgrad, using a 64-value transform in their main experiments [44]. NVIDIA instead transforms Wgrad only, using H16 for NVFP4 and H32 in its MXFP4 comparison. Its recipe specifies one sign vector shared across linear layers throughout training; the TE implementation used in our experiments stores a fixed 16-entry vector and caches the resulting transform [31, 32]. H16 and H32 name the number of values mixed by the transform.

Our kernels reuse that TE 16-entry sign vector as an explicit route constant. H16 applies the sign pattern once; H32 repeats it across two 16-value halves before the cross-half butterfly. No signs are drawn at setup or during training. We therefore call these paths *fixed-sign H16/H32 preconditioning*, rather than randomized Hadamard preconditioning. The additions and subtractions mix the values;

the fixed signs choose which values are negated first. Applying the same transform to X and dY preserves the unquantized Wgrad dot product, and the stored weight is never transformed. A fixed D remains orthogonal and repeatable across layers, steps, and resumes. It does not inherit a probabilistic concentration guarantee that assumes D is sampled independently of its input. We neither claim that guarantee nor compare alternative sign vectors.

As in NVIDIA’s recipe, Hadamard preconditioning remains Wgrad-only. Our backward producer creates the two consumer-specific representations in one boundary: row-facing dY uses SR for Dgrad, while deterministic column-facing dY is paired with transformed X for Wgrad. In the signed routes, the fixed-sign transform is performed in registers as part of column quantization rather than as a separate transform and repack pass. Dgrad and the stored weight are not transformed. H16 matches the 16-value NVFP4 scale block, while H32 matches the 32-value MXFP4 scale block [9]. The literal sign pattern and its half-block construction are recorded in Appendix A. Table 5 states the sign and rounding policy for every route.

2.6 Reference NVFP4 recipe versus the routes evaluated here

The NVIDIA recipe is a bundle, not only the NVFP4 datatype. It recommends 2D16 weights, SR on gradient inputs to Dgrad and Wgrad, a paired H16 signed Hadamard transform (described as RHT) for Wgrad, and retaining roughly 15% of empirically sensitive linears in higher precision, with most selected linears near the end of the network [32]. Three distinctions matter in this report. First, our TE-native comparator enables TE’s 2D, gradient-SR, and signed-Hadamard mechanisms but places all 32 body blocks in NVFP4; only the common output head is BF16. Second, the TE comparator with four final BF16 blocks is a separate NVIDIA-inspired route. It is not a reproduction of NVIDIA’s per-linear sensitivity list. Third, the custom global NVFP4, MXFP4, and CTA-local NVFP4 long routes use row-Dgrad SR, and only explicitly named routes apply Hadamard preconditioning. Tables 4 and 5 state the complete contract for each result.

3 Fused Execution

3.1 Fuse the complete producer–consumer boundary

Quantization helps only when its preparation work overlaps the matrix multiplication rather than extending the critical path. We model the latency of one FP4 boundary as

$$T_{\text{boundary}} \approx T_{\text{serial}} + \max(T_{\text{GEMM}}, T_{\text{reduce}} + T_{\text{quant}} + T_{\text{layout}}), \quad (11)$$

Here T_{serial} is work that cannot overlap because of a data dependency or resource contention. Fusion removes an HBM round trip only when the producer emits every scale, layout, and saved value required by the next operation. Accelerating only the GEMM epilogue can leave a global reduction, a second tensor orientation, or backward recomputation on the critical path. We implement the fused tensor operations in CUDA and ThunderKittens [39]. Following CODA’s GEMM-plus-epilogue framing [20], an operation joins a boundary only when the next stage can consume its output directly. A Llama block exposes four such boundaries: root-mean-square normalization (RMSNorm) into the query/key/value (QKV) projection, QKV into attention, the gated feed-forward network (FFN), and the residual update into the next RMSNorm. Each has a corresponding backward boundary.

Our FP4 boundary contract makes the fusion boundary explicit: a producer completes the group of values that share a scale, emits both physical orientations needed by Fprop, Dgrad, and Wgrad, and retains the BF16 and random state required by autograd. Scale completion, layout production, and saved-state requirements differ across MXFP4, custom global NVFP4, and CTA-local NVFP4, so one generic quantize-GEMM fusion schedule cannot represent all three paths. Figure 6 contrasts the serial boundaries with this format-aware production schedule.

3.2 What is fused in a Llama block

Figure 6 presents four execution panels: QKV forward and backward, and FFN forward and backward. The serial schedule writes BF16 intermediates to high-bandwidth memory (HBM), reads them back to compute scales and layouts, and repeats that cycle before the next GEMM. The fused schedule instead makes each producer emit the exact scale metadata, row and column views, and saved BF16 state that its consumers require.

For a fixed route, fusion preserves the defined numerical contract. It removes intermediate traffic and exposes independent work to the scheduler. The scale stage is nevertheless format-specific: MXFP4 completes a local 32-value reduction; custom global NVFP4 starts a tensor-wide maximum early; and CTA-local NVFP4 carries two tile outer scales through the complete contraction and applies their product in the GEMM epilogue.

The gated FFN illustrates why the boundary extends beyond a GEMM. It has gate and up projections (W_1 and W_3), followed by a down projection (W_2). Writing the W_1 and W_3 outputs to BF16, launching a separate SiLU-multiply, then reading the result again to quantize the W_2 input spends bandwidth between every large contraction. The fused path schedules the paired projections together and emits the FP4 input to W_2 with the SiLU-gate output. In backward, it combines the W_2 data gradient with the SiLU derivative and produces the representations consumed by the W_1 and W_3 data- and weight-gradient contractions. Exact BF16 values remain available where autograd requires them; fusion changes data movement, not the model definition.

QKV follows the same principle. The normalized input producer emits the format-specific representations consumed by the grouped projection, and the Q/K projection applies RoPE and the required layout. Backward combines inverse layout, RoPE, QKV gradient production, and the adjacent RMSNorm reduction when their data dependencies align. Residual-RMSNorm retains the exact residual and row statistics while producing the next quantized representations.

Scale work is scheduled by format. MXFP4 reductions finish with each local tile. Custom global NVFP4 begins the tensor maximum-absolute-value reduction early so scale finalization can overlap other work. CTA-local NVFP4 removes that global dependency, but its two outer scales must remain available through the full K reduction. Figure 2 summarizes these schedules.

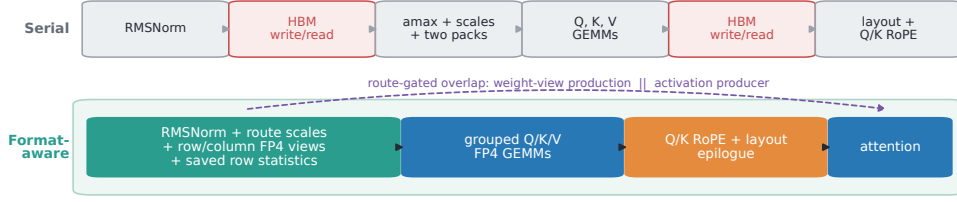
3.3 Depth-wise and operand-wise hybrids

We call the *depth-wise* route the 27/5 hybrid: it assigns complete transformer blocks 1–27 to CTA-local NVFP4 and blocks 28–32 to MXFP4. Attention and FFN linears follow the assigned block, so Fprop, Dgrad, and Wgrad never mix formats within one linear. Both formats use orientation-consistent 2D weights and row-only gradient-data SR; neither uses Hadamard preconditioning.

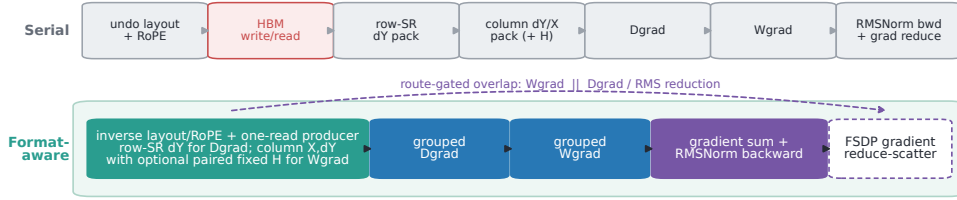
The *operand-wise* hybrid instead chooses the format per contraction. A controlled operator analysis using identical saved tensors found lower Dgrad error for CTA-local NVFP4 than for MXFP4 in

Fusing the repeated producer--consumer boundaries of a Llama block

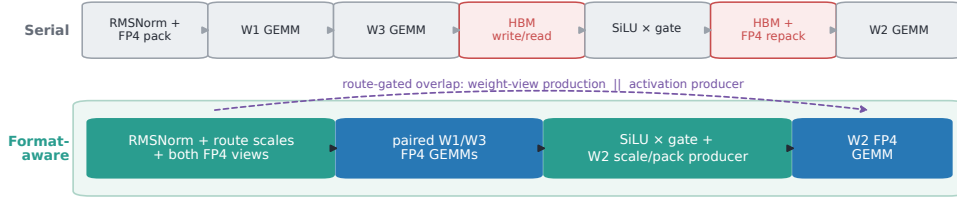
A QKV forward



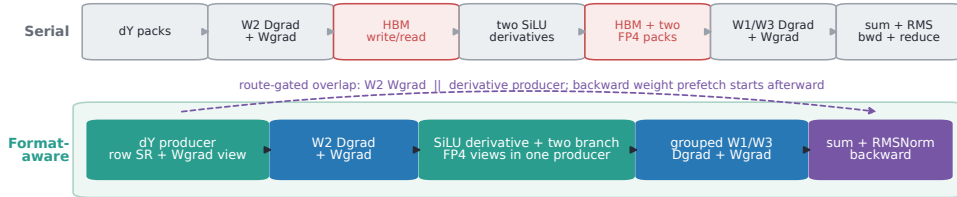
B QKV backward



C FFN forward



D FFN backward



Scale contract inside teal producers: MXFP4 completes local amax₃₂; global NVFP4 starts tensor amax early; CTA-local NVFP4 carries tile outer scales to the GEMM, whose epilogue applies α, β . Solid arrows are dependencies. Purple dashed arrows denote overlap, not kernel fusion. HBM boxes mark avoidable materialization.

Figure 6: Serial and format-aware execution of the QKV and gated-FFN boundaries. Solid boxes and arrows show values produced and consumed within one execution boundary; they do not imply one monolithic kernel. Purple dashed arrows show stream overlap. Distributed collectives remain separate operations even when their launch overlaps local work. In panels B and D, row stochastic rounding (SR) applies only to the gradient view consumed by Dgrad; a paired fixed-sign Hadamard transform (H), when enabled, applies to both column operands consumed by Wgrad.

every measured layer and component. This motivates replacing MXFP4 Dgrad with CTA-local NVFP4 while retaining the MXFP4 Fprop and Wgrad path. The diagnostic and its scope are reported in Section 5 and Appendix C.4.

Concretely, every linear uses MXFP4 for Fprop and Wgrad and CTA-local NVFP4 with row-data SR for Dgrad. One producer loads each BF16 gradient tile once and emits both the CTA-local row representation and the MXFP4 column representation. The weight producer likewise loads BF16 W once and emits the MXFP4 forward view and the CTA-local Dgrad view. The lower panel of Figure 4 shows this contraction-wise assignment for the plain-H16 variant.

We evaluate two Wgrad variants. The plain-H16 variant applies the same 16-value Hadamard mixing to X and dY with $D = I$, so it performs no sign flips. The fixed-H32 variant applies the paired 32-value signed transform used by the completed MXFP4+row-SR+fixed-H32 route. Neither variant transforms the stored weight. Fprop, CTA-local Dgrad, weight encodings, row SR, output projection, and loss retain the same numerical contract. Table 6 reports the two complete configurations; Appendix A gives the sign construction.

Every main custom route retains a BF16 output projection and compiled regular cross entropy. Appendix B summarizes the low-precision output heads that did not improve the end-to-end throughput–numerical tradeoff.

4 Experimental Setup

4.1 Model, data, and training horizon

The main workload is an 8B-parameter Llama-family decoder with RMSNorm, grouped-query attention, rotary position embeddings (RoPE), and a gated feed-forward network [16]. The sequence length is 8,192 and the global batch is 512 sequences, or 4,194,304 tokens per update. A 160B-token run ends at step 38,147. Unless stated otherwise, the named low-precision route applies to every body linear in all 32 transformer blocks.

All main custom routes use a BF16 output projection followed by Inductor-compiled regular cross entropy. The optimizer is AdamW with a 2,000-step linear warmup to a peak learning rate of 3×10^{-4} ; the rate remains constant after warmup over the reported horizon. Training losses are compared at matched optimizer steps. Each long-run recipe is represented by one trajectory, so the comparisons do not estimate seed-to-seed variation. Table 3 summarizes the shared configuration; the appendix records execution-specific details.

4.2 Recipe ledger

Tables 4 and 5 state the numerical recipe for every reported route. “Row SR” means stochastic rounding only for the row-oriented gradient consumed by Dgrad. Activations, weights, scales, and the separate Wgrad-oriented gradient representation remain deterministic in every custom route labelled “row SR.” A Wgrad Hadamard transform always acts on X and dY as a pair; it never transforms the stored weight.

“TE-managed” means that Transformer Engine controls the H16 transform. Its sign-state contract is distinct from the explicit fixed-sign contract used by the custom kernels.

All custom training routes retain BF16 parameters and FP32 optimizer state. The final column

Table 3: Shared Llama-8B training configuration.

Field	Effective contract
Model	32 blocks; width 4,096; FFN 14,336; 32 Q heads; 8 KV heads; head dimension 128; vocabulary 128,256
Position / norm	Llama-3.1 RoPE with base 500,000, $8\times$ scaling, low/high frequency factors 1 and 4, and original length 8,192; RMSNorm $\epsilon = 10^{-5}$
Data / tokenizer	Shuffled OLMo Mix 1124 [43]; Meta-Llama-3.1-8B tokenizer
Optimizer	Fused AdamW, $(\beta_1, \beta_2) = (0.9, 0.95)$, $\epsilon = 10^{-8}$, weight decay 0.1, global gradient-norm clip 1.0
LR schedule	2,000-step linear warmup to 3×10^{-4} ; constant thereafter over the reported horizon
Batch / horizon	Global batch 512; 4,194,304 tokens/update; 38,147 updates; 160,000,114,688 scheduled tokens
State / output	BF16 parameters and non-FP4 operations; FP32 master/optimizer state; BF16 output projection; execution-specific regular CE

Table 4: Per-contraction numerical recipe. 2D16 and 2D32 denote one orientation-consistent 16×16 or 32×32 weight encoding. NV denotes global NVFP4, LC denotes CTA-local NVFP4, and MX denotes MXFP4. The 27/5 hybrid changes format by depth; the operand hybrid changes it by contraction inside every block.

Route	Body assignment	Fprop	Dgrad	Wgrad	Weight representation(s)
BF16	blocks 1–32	BF16	BF16	BF16	BF16 master
TE-native NVFP4 in all blocks	blocks 1–32	global NV	global NV	global NV + TE-managed H16	TE 2D16
TE NVFP4 with four final BF16 blocks	NV blocks 1–28; BF16 29–32	NV 1–28; BF16 29–32	NV 1–28; BF16 29–32	NV+TE-managed H16 1–28; BF16 29–32	TE 2D16 / BF16
Custom global NVFP4	blocks 1–32	global NV	global NV	global NV	custom 2D16
MXFP4 + row-SR	blocks 1–32	MX	MX	MX	custom 2D32
CTA-local NVFP4	blocks 1–32	localCTA	localCTA	localCTA	custom 2D16
CTA-local NVFP4+fixed H16	blocks 1–32	localCTA	localCTA	localCTA + paired fixed-sign H16	custom 2D16
27/5 depth hybrid	LC blocks 1–27; MX 28–32	LC 1–27; MX 28–32	LC 1–27; MX 28–32	LC 1–27; MX 28–32	LC 2D16; MX 2D32
MXFP4+row-SR+fixed H32	blocks 1–32	MX	MX	MX + paired fixed-sign H32	custom 2D32
Operand hybrid (plain H16)	every block	MX	localCTA	MX + paired plain H16	MX 2D32 + LC 2D16 encodings
Operand hybrid (fixed H32)	every block	MX	localCTA	MX + paired fixed-sign H32	MX 2D32 + LC 2D16 encodings

Table 5: Rounding and precision policy. Every row uses a BF16 output projection.

Route	Gradient rounding	Wgrad preconditioning	BF16 body blocks / output head
BF16	n/a	none	all / BF16
TE-native NVFP4 in all blocks	TE gradient SR	TE-managed H16 on Wgrad	none / BF16
TE NVFP4 + four final BF16 blocks	TE gradient SR	TE-managed H16 on NV Wgrad	last four / BF16
Custom global NVFP4	row data SR	none	none / BF16
MXFP4 + row-SR	row data SR	none	none / BF16
CTA-local NVFP4	row data SR	none	none / BF16
CTA-local NVFP4+fixed H16	row data SR	paired fixed-sign H16 on Wgrad X, dY	none / BF16
27/5 depth hybrid	row data SR in both	none	none / BF16
MXFP4+row-SR+fixed H32	row data SR	paired fixed-sign H32 on Wgrad X, dY	none / BF16
Operand hybrid (plain H16)	LC row data SR	plain H16 ($D = I$) on Wgrad	none / BF16
Operand hybrid (fixed H32)	LC row data SR	paired fixed-sign H32 on Wgrad X, dY	none / BF16

in Table 4 describes the transient FP4 representation consumed by Tensor Cores. The operand hybrid emits two encodings from one BF16 weight: MX 2D32 for Fprop and CTA-local 2D16 for Dgrad. Wgrad does not consume the stored weight; it quantizes X and dY with MXFP4. This is a deliberate contraction-specific recipe rather than two layouts of one shared FP4 weight.

Every operand marked “MX” in Table 4 uses encode-centric E8M0 scales for activation and gradient blocks and the adaptive 2D32 weight rule in Section 2.2. The scale exponent itself is deterministic. “Row SR” applies to the E2M1 gradient values, not to their E8M0 scale. This distinction is part of the recipe: scale direction controls range and grid spacing, whereas data SR controls how a value is selected between adjacent E2M1 levels.

Appendix C gives the checkpoint inclusion criteria and additional reproducibility details.

4.3 End-to-end performance measurements

The performance probes use the same accelerator type, sequence length, global batch, seed, BF16 output projection, compiled CE, warmup rule, metric window, and per-GPU timing records. Each route uses its selected local batch, activation-checkpoint policy, and distributed schedule, so the panel compares complete end-to-end configurations rather than one isolated implementation change. Tables report cross-device aggregates. Local batch (LB) and gradient accumulation (GA) remain explicit because they affect occupancy, memory lifetime, and the exposed output-layer cost.

We use the dense-model training FLOP convention implemented by TorchTitan. For total parameter count N , embedding parameter count N_{emb} , number of layers L , query-head count H , head dimension d_h , and sequence length S , it gives

$$F_{\text{model/token}} = 6(N - N_{\text{emb}}) + 12LHd_hS. \quad (12)$$

The first term counts the forward and backward dense parameter operations; the second counts the attention contractions without charging recomputation. Here $N = 8,030,261,248$, $N_{\text{emb}} =$

525,336,576, $L = 32$, $H = 32$, $d_h = 128$, and $S = 8,192$, giving $F_{\text{model/token}} = 57,914,449,920$ FLOPs/token.

For a route with per-device token rate R , we report BF16 model FLOP utilization (BF16 MFU):

$$\text{MFU}_{\text{BF16}} = 100 \frac{F_{\text{model/token}} R}{F_{\text{peak,BF16}}}, \quad (13)$$

using Equation 12 and the hardware’s maximum dense BF16 Tensor Core throughput. NVIDIA specifies 360 PFLOP/s of sparse FP16/BF16 throughput for a 72-GPU GB200 NVL72 and states that dense throughput is half the sparse value. Dividing by 72 GPUs and by two gives the maximum dense BF16 throughput used here, $F_{\text{peak,BF16}} = 2.5 \times 10^{15}$ FLOP/s/GPU, or 2,500 BF16 TFLOP/s/GPU [29]. Tokens/s/GPU is the directly observed metric. We recompute BF16 MFU from each unrounded throughput aggregate with this peak. For an FP4 route, BF16 MFU is BF16-equivalent model work divided by the BF16 peak; it measures effective training throughput, not literal FP4 Tensor Core occupancy. It can therefore exceed 100%. Recomputing from the rounded token rate printed in a table can differ in the last decimal place.

4.4 Training and checkpoint evaluation

Training curves use exact logged optimizer steps. Smoothed curves use an exponential moving average only for visualization. Custom-route endpoints are raw observations; Appendix C defines the BF16 and TE-native reference curves.

Validation uses a fixed external stream constructed from an 82/18 DataComp-LM (DCLM) and OLMo-without-DCLM source mixture [26, 43]. It contains 768 sequences and 6,291,456 next-token targets. Documents are deduplicated, mixed, and shuffled before packing so one GPU worker cannot receive examples from only one source or contiguous region of source data. Every checkpoint consumes the same tokenized sequences. We call this an external validation stream rather than a held-out split because zero overlap with every historical training trajectory cannot be proven. Before route comparison, the converted-checkpoint evaluator and native TorchTitan model must agree at the full 8,192-token context, and a late BF16 checkpoint must improve over an early one by more than five sequence-level standard errors.

Every downstream comparison uses the literal step-38,000 checkpoint and BF16 inference. Converted models must match TorchTitan’s RoPE and RMSNorm semantics before scoring. The evaluation panel follows the four-task subset used by the reference NVFP4 protocol: MMLU, HellaSwag, WinoGrande, and ARC-Challenge [19, 22, 32]. Shot counts and reported metrics appear with the results.

5 Results

5.1 What throughput do the end-to-end routes achieve?

The fastest custom path, MXFP4 with row-gradient SR, reaches 37,876.8 tokens/s/GPU, compared with 18,812.0 for BF16 and 27,622.0 for TE-native NVFP4 in all blocks. Table 6 reports the complete same-accelerator panel. Every row uses sequence length 8,192, global batch 512, a BF16 output projection, compiled regular CE, and a cross-device median over steps 40–100. BF16 and TE use local batch size 2 with eight gradient-accumulation steps (LB2/GA8); the custom paths use local batch size 4 with four gradient-accumulation steps (LB4/GA4).

Table 6: GB200 throughput with compiled regular CE. Values are cross-device medians over steps 40–100 for each route’s selected batch geometry. BF16 uses the selective-8 setting, which checkpoints every eighth transformer block; all other rows use selective-32, which checkpoints every 32nd block. BF16 MFU uses NVIDIA’s 2,500-TFLOP/s/GPU maximum dense BF16 throughput: 180 PFLOP/s dense for 72 GPUs [29].

Route	LB/GA	Tokens/s/GPU	BF16 MFU	Median HBM
BF16	2/8	18,812.00	43.58%	101.19 GiB
TE-native NVFP4 in all blocks	2/8	27,622.00	63.99%	125.80 GiB
TE NVFP4 + four final BF16 blocks	2/8	27,077.50	62.73%	126.03 GiB
Custom global NVFP4	4/4	32,490.50	75.27%	155.63 GiB
MXFP4 + row-SR	4/4	37,876.75	87.74%	154.43 GiB
CTA-local NVFP4	4/4	33,243.76	77.01%	115.52 GiB
CTA-local NVFP4 + fixed H16	4/4	31,882.61	73.86%	114.89 GiB
27/5 depth hybrid	4/4	33,120.50	76.73%	126.21 GiB
Operand hybrid + plain H16	4/4	34,782.50	80.58%	154.92 GiB
Operand hybrid + fixed H32	4/4	35,085.50	81.28%	150.89 GiB
MXFP4 + row-SR + fixed H32	4/4	37,247.00	86.29%	151.02 GiB

Under the same 32-GPU geometry with local batch size 4 and four gradient accumulation steps (LB4/GA4), the MXFP4+row-SR+fixed-H32 route reaches 98.34% of the row-SR-only MXFP4 throughput. Additional measurement details appear in Appendix A.

5.2 How closely do the recipes track BF16 training loss?

Figure 7 compares the training trajectories. The top panel shows an exponential moving average over 31 logged observations (EMA-31). The bottom panel follows the relative-difference convention used by NVIDIA [32]:

$$\Delta_{\text{BF16}}(t) = 100 \frac{L_{\text{BF16}}(t) - L_{\text{route}}(t)}{L_{\text{BF16}}(t)}. \quad (14)$$

Zero is the BF16 reference and a negative value means the low-precision route has higher loss. Reported comparisons use exact shared logged steps; no reported loss value is interpolated. The dashed custom-global-NVFP4 line is the display-only fit described below.

Figure 8 expands the final 40B-token window. It shows the late loss separation at the same scale without extrapolating any route.

The TE route with four final BF16 blocks and the fixed-H32 operand hybrid both complete the scheduled training horizon.

Llama-8B: training loss and relative difference from BF16

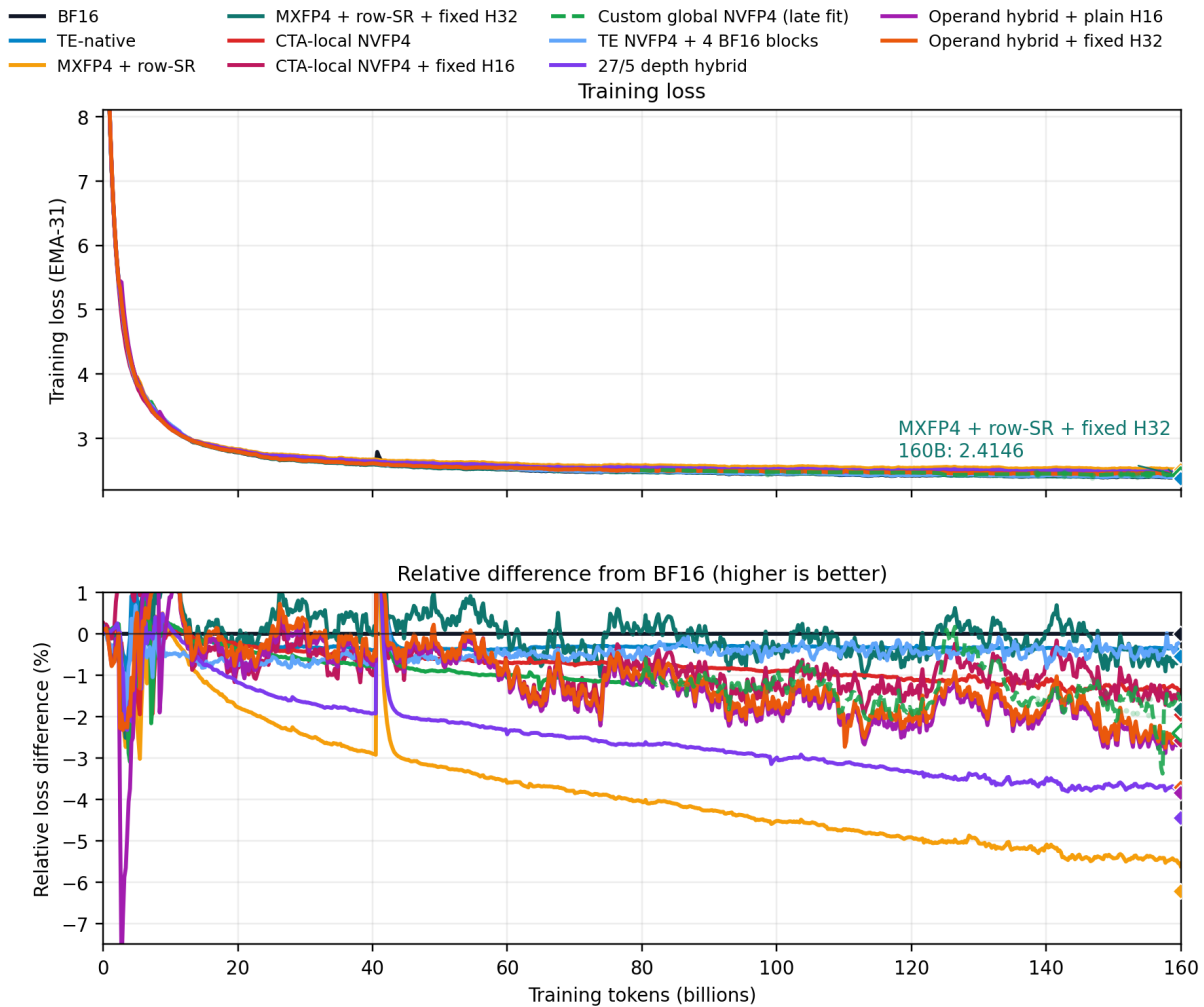


Figure 7: Llama-8B training loss and relative difference from the BF16 reference. Continuous histories use EMA-31, and diamonds mark the final plotted endpoint. Custom-global-NVFP4 has regular observations through step 19,000, but its dense later history was not retained. The upper panel therefore shows the 74 exact later observations recovered from training logs, including the terminal diamond; the lower panel shows the 18 observations recorded at the same step as BF16. The dashed custom-global-NVFP4 line is a smoothed visual guide through those sparse observations, not additional measurements. Appendix C records the surviving data sources.

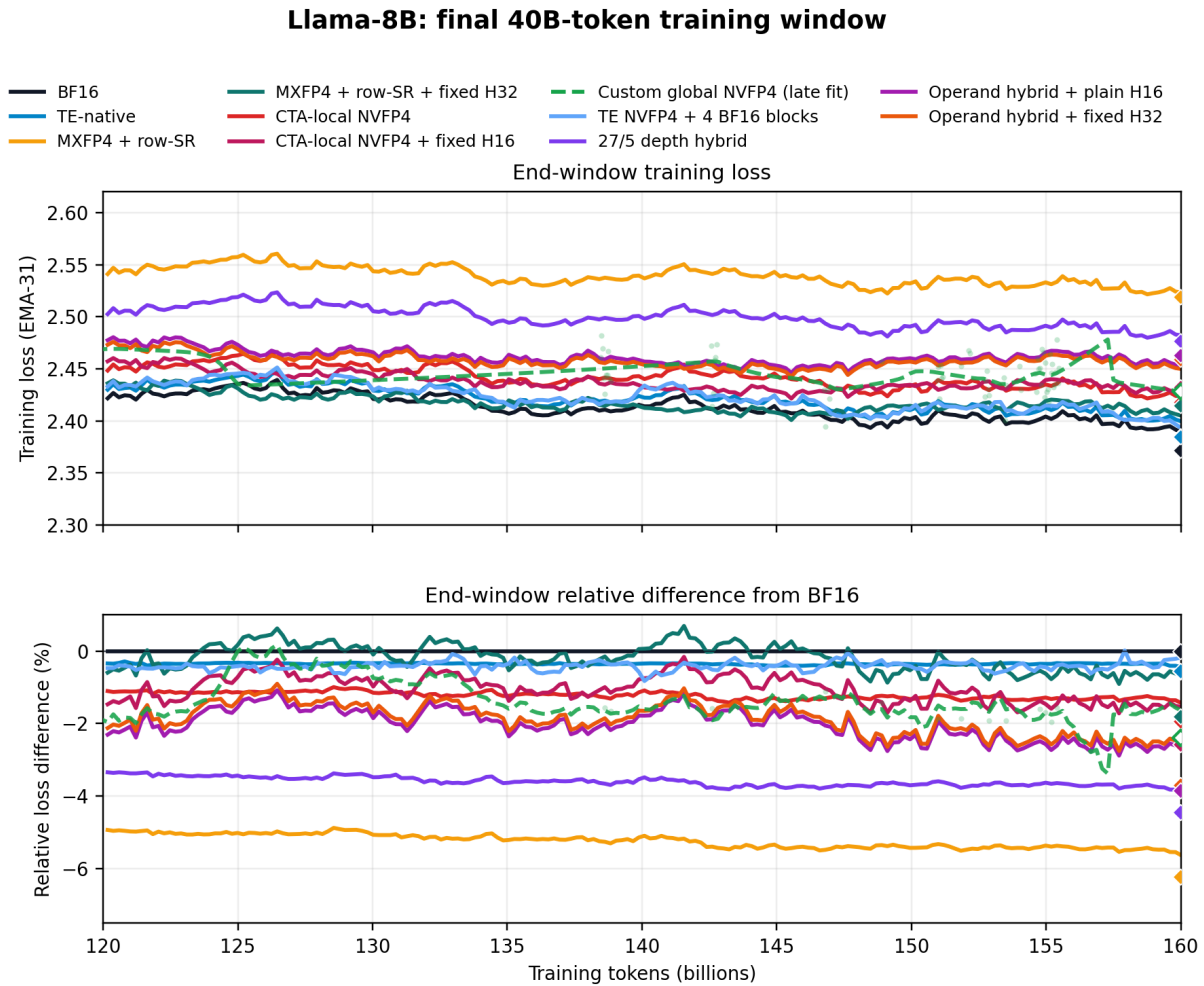


Figure 8: Training loss from 120B to 160B tokens. Diamonds mark exact logged endpoints. As in Figure 7, the late custom-global-NVFP4 markers are exact observations and the dashed curve is a smoothed visual guide, not additional measurements.

Table 7: Raw last logged minibatch loss at step 38,140 (159.97B tokens). “Loss excess” is relative to the unadjusted BF16 observation at that step.

Route	Raw endpoint loss	Loss excess vs BF16
BF16	2.364637	0.00%
TE NVFP4 + four final BF16 blocks	2.385181	0.87%
TE-native NVFP4	2.396353	1.34%
MXFP4 + row-SR + fixed H32	2.414578	2.11%
CTA-local NVFP4	2.417708	2.24%
Custom global NVFP4	2.428200	2.69%
CTA-local NVFP4 + fixed H16	2.432642	2.88%
Operand hybrid + fixed H32	2.459305	4.00%
Operand hybrid + plain H16	2.462673	4.15%
27/5 depth hybrid	2.476842	4.75%
MXFP4 + row-SR	2.519150	6.53%

At step 38,140, the TE route with four final BF16 blocks is the closest quantized trajectory to the raw BF16 observation, with 0.87% excess, followed by TE-native at 1.34%. MXFP4 with row-SR and fixed H32 is the closest custom route at 2.11%; it is 0.76% above TE-native. The fixed-H32 operand hybrid ends 1.85% above the pure MXFP4-H32 route and 4.00% above BF16.

To compare error in the backward input-gradient calculation (Dgrad), we used tensors captured at step 28,000 from the MXFP4+row-SR+fixed-H32 run. We recomputed Dgrad with CTA-local NVFP4 and with MXFP4, using BF16 as the reference. CTA-local NVFP4 had lower Dgrad error energy in all 288 layer–component comparisons; the geometric-mean $E_{MX}/E_{localCTA}$ ratio was 1.292. The completed plain-H16 operand hybrid nevertheless ends 1.99% above MXFP4+row-SR+fixed H32 in training loss. Appendix C.4 gives the capture and aggregation contract.

5.3 Validation loss

Figure 9 evaluates every available exact checkpoint on one fixed external stream. We report mean negative log-likelihood (NLL), for which lower values are better. At the common step-38,000 cut, MXFP4 with fixed H32 has the lowest observed validation NLL, 2.422804. BF16 records 2.430206, the TE route with four final BF16 blocks 2.437475, and TE-native 2.440488. The fixed-H32 operand hybrid records 2.463842, compared with 2.460331 for CTA-local NVFP4 and 2.462074 for custom global NVFP4. Relative to BF16, the relative NLL differences $100(L_{BF16} - L_{route})/L_{BF16}$ for MXFP4-H32, four-final-BF16-block, and operand-H32 are +0.305%, −0.299%, and −1.384%, respectively; positive values indicate lower NLL than BF16. These comparisons cover complete recipes from one training trajectory each; they do not isolate the effect of H32 or average over repeated seeds.

The 27/5 depth hybrid is evaluated at the five common cuts: steps 2,000, 10,000, 18,000, 29,000, and 38,000. At step 38,000 its validation NLL is 2.538116, or 4.440% higher than matched BF16.

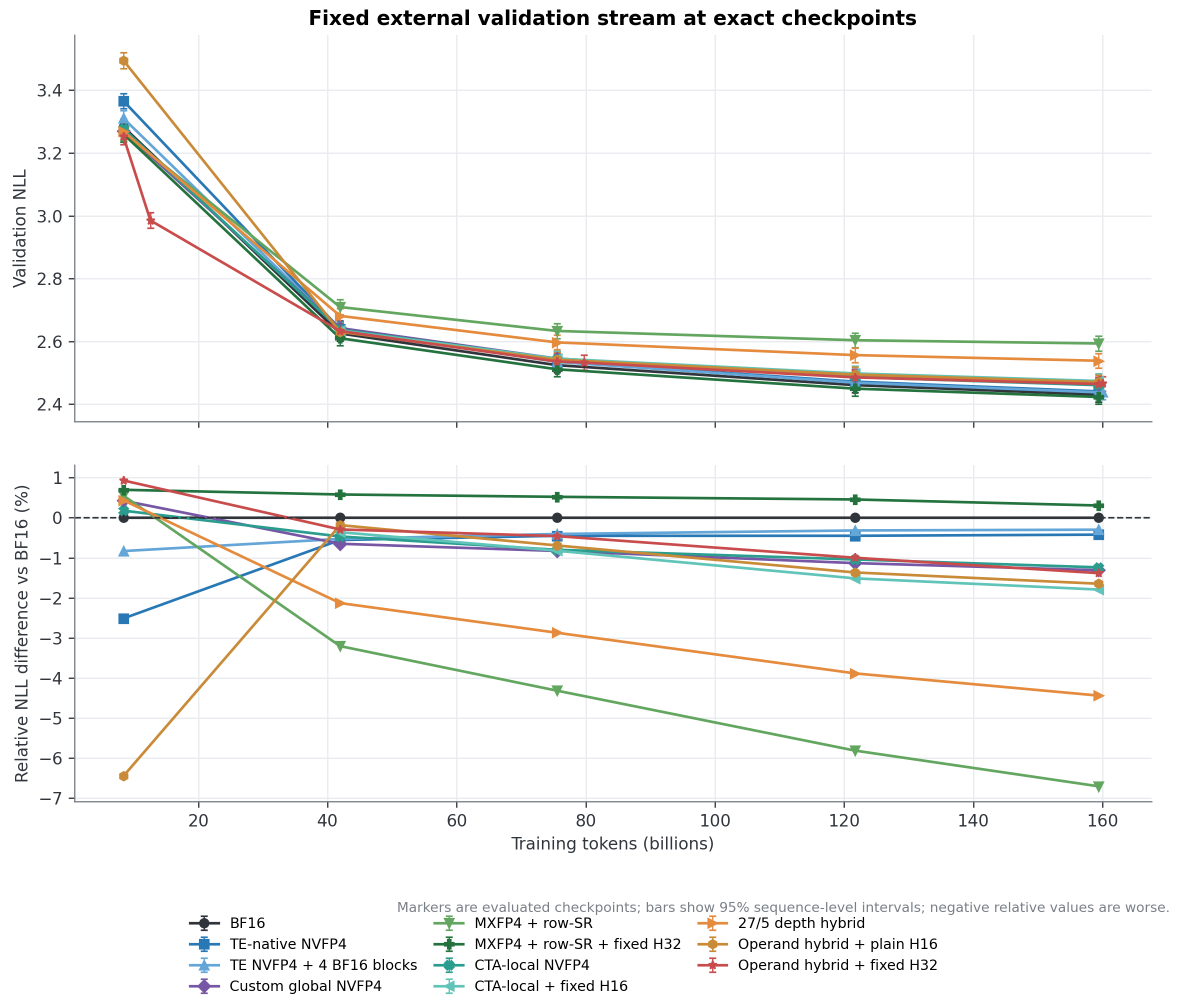


Figure 9: Validation NLL on the fixed external stream at every available exact checkpoint. Error bars are 95% sequence-level intervals. The lower panel reports $100(L_{\text{BF16}} - L_{\text{route}})/L_{\text{BF16}}$ at exact shared steps, so negative values are worse than BF16. Lines connect evaluated checkpoints only; no value is interpolated or extrapolated.

5.4 How do the routes compare on downstream tasks?

The common panel evaluates all eleven routes with a complete exact step-38,000 checkpoint. Every route uses BF16 inference and the same MMLU, HellaSwag, WinoGrande, and ARC-Challenge protocol.

Table 8: Exact step-38,000 downstream scores (%). MMLU and WinoGrande report accuracy; HellaSwag and ARC-Challenge report normalized accuracy. Shot counts are 5, 10, 5, and 25, respectively. Every route uses BF16 inference after exact converted/native parity. Bold marks the highest observed score in each column.

Route	MMLU	HellaSwag	WinoGrande	ARC-Challenge
BF16	28.64	67.50	62.12	42.06
TE-native NVFP4	25.19	66.62	63.14	42.49
TE NVFP4, four final BF16 blocks	26.45	66.97	61.01	43.26
Custom global NVFP4	27.38	65.81	63.85	41.72
MXFP4 + row-SR	25.94	60.39	60.69	37.54
MXFP4 + row-SR + fixed H32	25.47	68.82	63.61	44.54
CTA-local NVFP4	25.54	66.32	62.75	42.66
CTA-local NVFP4 + fixed H16	23.74	65.08	63.06	43.34
27/5 depth hybrid	25.16	62.94	61.17	39.08
Operand hybrid + plain H16	24.40	66.47	62.98	42.92
Operand hybrid + fixed H32	23.81	66.80	64.40	42.15

The task ordering is not uniform. BF16 has the highest MMLU score, the fixed-H32 operand hybrid has the highest WinoGrande score, and MXFP4+row-SR+fixed H32 has the highest HellaSwag and ARC-Challenge scores. Relative to that pure MXFP4-H32 route, the fixed-H32 operand hybrid differs by -1.66 , -2.02 , $+0.79$, and -2.39 points on MMLU, HellaSwag, WinoGrande, and ARC-Challenge, respectively. The four-final-BF16-block TE route differs from BF16 by -2.19 , -0.53 , -1.10 , and $+1.19$ points. These are observations from one checkpoint per trajectory, not repeated-seed estimates.

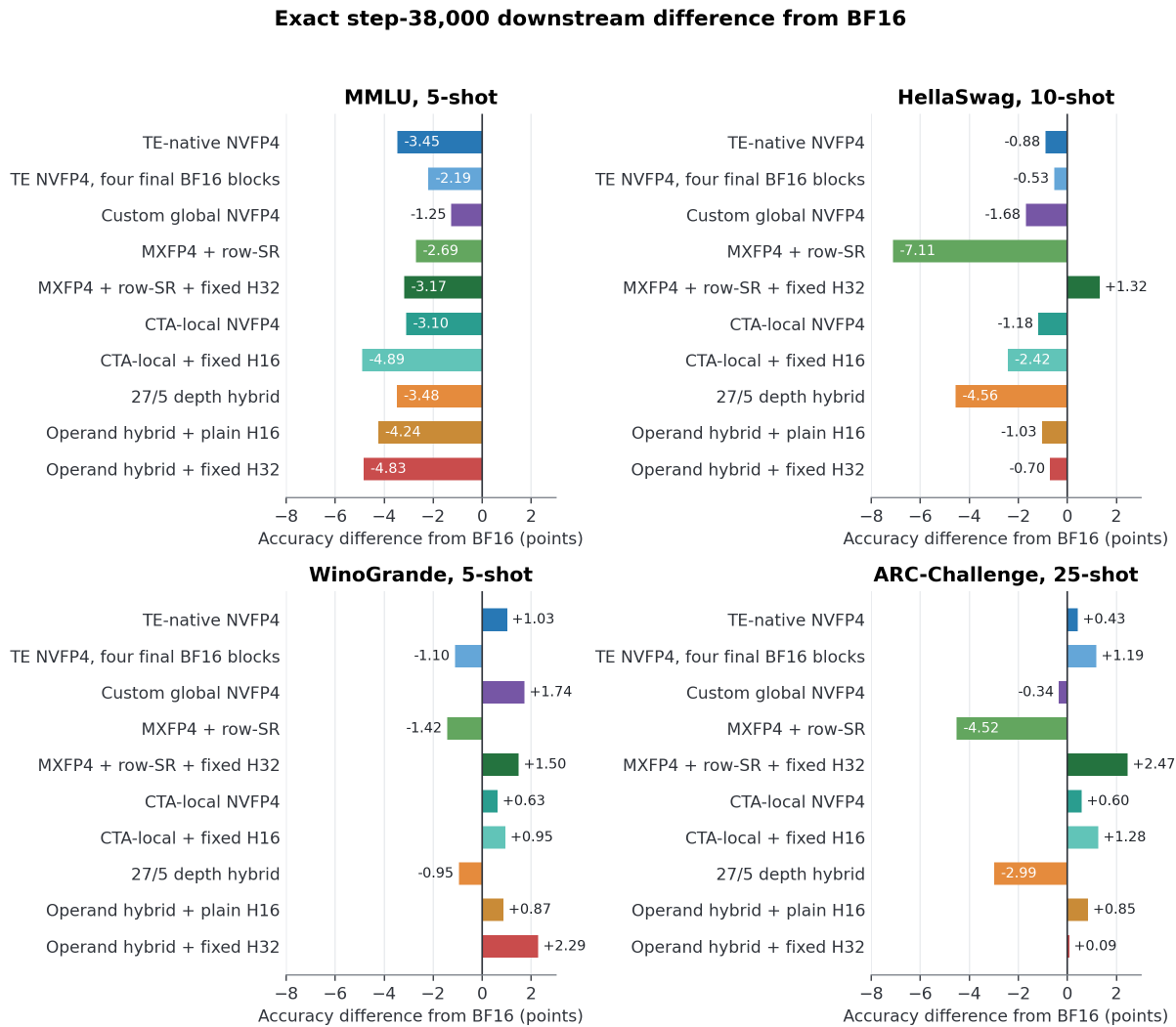


Figure 10: Downstream reported-score difference from BF16 at exact step 38,000. Positive values exceed the BF16 score on that task. All panels use the same percentage-point scale. The absolute scores are reported in Table 8.

6 Discussion

The measured TE routes track BF16 closely: TE-native and the four-final-BF16-block route end 1.34% and 0.87% above the raw BF16 loss. Their common probes reach 27.6K and 27.1K tokens/s/GPU. A separate same-accelerator probe of the custom global NVFP4 route reaches 32.5K tokens/s/GPU; its long trajectory ends 2.69% above BF16. TE-native NVFP4 and custom global NVFP4 both use the tensor-global NVFP4 scale hierarchy; the four-final-BF16-block route uses that hierarchy in its first 28 blocks. The panel does not isolate the cost of scaling from implementation and batch geometry, but it exposes the systems constraint: a tensor-global maximum-absolute-value reduction creates a completion dependency that the execution path must hide or pay.

Hadamard preconditioning is not uniformly beneficial across scale strategies. The fixed-H32 MXFP4 recipe retains 98.34% of row-SR-only MXFP4 throughput while moving the raw endpoint excess from 6.53% to 2.11% and validation NLL from 2.593146 to 2.422804. The fixed-H16 CTA-local recipe is slower than CTA-local without the transform and has higher endpoint and validation loss. These are comparisons between complete trajectories, not experiments that isolate the effect of Hadamard preconditioning alone.

Neither tested hybrid dominates the strongest pure custom route. The depth-wise hybrid and both operand-wise hybrids are slower and have higher endpoint and validation loss than MXFP4 with row-SR and fixed H32. A controlled replay on identical saved tensors still finds lower Dgrad error for CTA-local in all 288 tested layer-projection comparisons. The completed hybrid trajectories do not preserve that operator-level advantage in endpoint or validation loss.

7 Conclusion and Further Work

Format-aware fusion makes the complete FP4 producer-consumer boundary the unit of optimization. The TE routes give the closest observed training-loss tracking, with measured tensor-global execution paths at 27.1–27.6K tokens/s/GPU. Among the custom routes, MXFP4 with row-gradient SR and fixed-sign H32 Wgrad preconditioning reaches 37.2K tokens/s/GPU, 86.3% BF16 MFU, a 2.11% raw endpoint-loss excess, and the lowest observed validation NLL on the fixed external data stream. Its downstream ranking remains task dependent. BF16 records the highest MMLU score, the fixed-H32 operand hybrid the highest WinoGrande score, and MXFP4 with row-SR and fixed H32 the highest HellaSwag and ARC-Challenge scores. In this panel, late training loss and downstream accuracy therefore do not have a monotonic ordering; training loss alone does not rank the downstream quality of these complete low-precision recipes.

Neither the tested depth-wise nor operand-wise hybrid improves the combined speed-quality point of the strongest pure custom route. The opposite outcomes of Hadamard preconditioning under MXFP4 and CTA-local scaling motivate choosing the group of values sharing each scale, the rounding rule, and the preconditioner jointly. A complementary direction is to change the scale representation itself. UE5M3 FP4 pairs E2M1 payloads with an unsigned E5M3 block scale and has supported a recipe with periodic tensor-scale updates, selective SR, and no Hadamard preconditioning on a separate model and training setup [25]. Combining wider unsigned block scales with a CTA-local producer schedule, or with delayed tensor scaling and global outer scales, is a natural next step.

Generative-AI Disclosure

Generative AI tools assisted with code development, experiment orchestration, analysis, figure generation, and drafting and editing this report. The author reviewed and verified the code, experiments, numerical results, figures, citations, and final text and takes responsibility for the content.

References

- [1] Anjuli Agrusa, Andrei Panferov, Elizabeth Wei, Keith Wyss, Paul Gibbons, Erik Schultheis, Tijmen Blankevoort, and Dan Alistarh. What matters for NVFP4 training? a scaling study of low-precision pre-training recipes. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL <https://openreview.net/forum?id=jlklIyaG32w>.
- [2] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. QuaRot: Outlier-free 4-bit inference in rotated LLMs. *arXiv preprint arXiv:2404.00456*, 2024. doi: 10.48550/arXiv.2404.00456. URL <https://arxiv.org/abs/2404.00456>.
- [3] Hengjie Cao, Mengyi Chen, Yifeng Yang, Ruijun Huang, Fang Dong, Jixian Zhou, et al. Metis: Training LLMs with FP4 quantization. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=I2ZrCi5084>.
- [4] Roberto L. Castro, Andrei Panferov, Rush Tabesh, Oliver Sieberling, Jiale Chen, Mahdi Nikdan, Saleh Ashkboos, and Dan Alistarh. Quartet: Native FP4 training can be optimal for large language models. In *Advances in Neural Information Processing Systems*, volume 38, pages 43552–43572, 2025. doi: 10.52202/085713-1449. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/3e11872d3bc9bfba34e1752fef87639c-Abstract-Conference.html.
- [5] Yuxiang Chen, Haocheng Xi, Jun Zhu, and Jianfei Chen. Oscillation-reduced MXFP4 training for vision transformers. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 9400–9414, 2025. URL <https://proceedings.mlr.press/v267/chen25bu.html>.
- [6] Yuxiang Chen, Yifan Liu, Xiaoming Xu, Pengle Zhang, Michael Beyer, Martin Rapp, Jun Zhu, and Jianfei Chen. TetraJet-v2: Accurate NVFP4 training for large language models with oscillation suppression and outlier control. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL <https://arxiv.org/abs/2510.27527>.
- [7] Brian Chmiel, Ron Banner, Elad Hoffer, Hilla Ben Yaacov, and Daniel Soudry. Accurate neural training with 4-bit matrix multiplications at standard formats. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2112.10769>.
- [8] Brian Chmiel, Maxim Fishman, Ron Banner, and Daniel Soudry. FP4 all the way: Fully quantized training of large language models. In *Advances in Neural Information Processing Systems*, volume 38, pages 91130–91151. Curran Associates, Inc., 2025. doi: 10.52202/085713-3049. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/8340b085045cf13f1f0b6c2c4cc0a89c-Paper-Conference.pdf.

- [9] Musa Cim, Sarthak Arora, Poovaiah Palangappa, Miro Hodak, Ravi Dwivedula, Meena Arunachalam, and Mahmut Taylan Kandemir. Pretraining large language models with MXFP4 on native FP4 hardware. *arXiv preprint arXiv:2605.09825*, 2026. doi: 10.48550/arXiv.2605.09825. URL <https://arxiv.org/abs/2605.09825>. Version 4, revised 12 August 2026.
- [10] Jack Cook, Junxian Guo, Guangxuan Xiao, Yujun Lin, Keith Wyss, Mahdi Nazemi, Asit Mishra, Carlo del Mundo, Tijmen Blankevoort, and Song Han. Four over six: More accurate NVFP4 quantization with adaptive block scaling. *arXiv preprint arXiv:2512.02010*, 2025. doi: 10.48550/arXiv.2512.02010. URL <https://arxiv.org/abs/2512.02010>.
- [11] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023. URL <https://arxiv.org/abs/2307.08691>.
- [12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Re. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL <https://arxiv.org/abs/2205.14135>.
- [13] Siyu Ding, Mingchuan Ma, Jiabo Tong, Xingrun Xing, Ziming Wang, and Guoqi Li. Full-stack FP4: Stable LLM pretraining with quantized projections, optimizers, and attention. *arXiv preprint arXiv:2607.04422*, 2026. doi: 10.48550/arXiv.2607.04422. URL <https://arxiv.org/abs/2607.04422>.
- [14] Peijie Dong, Ruibo Fan, Yuechen Tao, Di Mou, Wenhui Hu, Zhenheng Tang, Yinghao Yu, Jiamang Wang, Wenbo Su, Guodong Yang, Liping Zhang, Xiaowen Chu, Baochun Li, and Bo Li. Dissecting outlier dynamics in LLM NVFP4 pretraining. *arXiv preprint arXiv:2602.02047*, 2026. URL <https://arxiv.org/abs/2602.02047>.
- [15] Jinyang Du, Ruihao Gong, Linghan Ai, Zining Wang, Yunke Peng, Yao Wang, Lei Yan, et al. Half-s: Halving the scale for near-lossless 4-bit LLM training. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 4890–4903, 2026. doi: 10.18653/v1/2026.findings-acl.241. URL <https://aclanthology.org/2026.findings-acl.241/>.
- [16] Abhimanyu Dubey et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL <https://arxiv.org/abs/2407.21783>.
- [17] Andrea Fasoli, Monodeep Kar, Chi-Chun Liu, Swagath Venkataramani, Viji Srinivasan, Leland Chang, and Naigang Wang. Is finer better? the limits of microscaling formats in large language models. *arXiv preprint arXiv:2601.19026*, 2026. doi: 10.48550/arXiv.2601.19026. URL <https://arxiv.org/abs/2601.19026>.
- [18] Andrew W. Fitzgibbon and Stephen Felix. On stochastic rounding with few random bits. In *2025 IEEE 32nd Symposium on Computer Arithmetic (ARITH)*, pages 133–140, 2025. doi: 10.1109/ARITH64983.2025.00029. URL <https://doi.org/10.1109/ARITH64983.2025.00029>.
- [19] Yuling Gu, Oyvind Tafjord, Bailey Kuehl, Dany Haddad, Jesse Dodge, and Hannaneh Hajishirzi. OLMES: A standard for language model evaluations. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 5020–5048. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-naacl.282. URL <https://aclanthology.org/2025.findings-naacl.282/>.
- [20] Han Guo, Jack Zhang, Arjun Menon, Driss Guessous, Vijay Thakkar, Yoon Kim, and Tri Dao. CODA: Rewriting transformer blocks as GEMM-epilogue programs. *arXiv preprint arXiv:2605.19269*, 2026. URL <https://arxiv.org/abs/2605.19269>.

- [21] Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. Deep learning with limited numerical precision. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1737–1746, 2015. URL <https://proceedings.mlr.press/v37/gupta15.html>.
- [22] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021. URL <https://arxiv.org/abs/2009.03300>.
- [23] Pin-Lun Hsu et al. Liger kernel: Efficient Triton kernels for LLM training. *arXiv preprint arXiv:2410.10989*, 2024. URL <https://arxiv.org/abs/2410.10989>.
- [24] Robert Hu, Carlo Luschi, and Paul Balanca. Elucidating the design space of FP4 training. *arXiv preprint arXiv:2509.17791*, 2025. doi: 10.48550/arXiv.2509.17791. URL <https://arxiv.org/abs/2509.17791>.
- [25] Robert Hu, Carlo Luschi, and Paul Balanca. UE5M3 FP4 block scaling for stable language model pretraining. *arXiv preprint arXiv:2609.02846*, 2026. doi: 10.48550/arXiv.2609.02846. URL <https://arxiv.org/abs/2609.02846>.
- [26] Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, et al. DataComp-LM: In search of the next generation of training sets for language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-0455. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/19e4ea30dded58259665db375885e412-Abstract-Datasets_and_Benchmarks_Track.html.
- [27] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=r1gs9JgRZ>.
- [28] Paulius Micikevicius, Dusan Stolic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, et al. FP8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022. doi: 10.48550/arXiv.2209.05433. URL <https://arxiv.org/abs/2209.05433>.
- [29] NVIDIA. NVIDIA GB200 NVL72, 2025. URL <https://www.nvidia.com/en-us/data-center/gb200-nvl72/>.
- [30] NVIDIA. NVIDIA HGX platform, 2026. URL <https://www.nvidia.com/en-us/data-center/hgx/>.
- [31] NVIDIA. NVFP4 training with transformer engine, 2026. URL https://docs.nvidia.com/deeplearning/transformer-engine-releases/release-2.15/user-guide/features/low_precision_training/nvfp4/nvfp4.html.
- [32] NVIDIA, Felix Abecassis, Anjulie Agrusa, Dong Ahn, Jonah Alben, Stefania Alborghetti, et al. Pretraining large language models with NVFP4. *arXiv preprint arXiv:2509.25149*, 2025. doi: 10.48550/arXiv.2509.25149. URL <https://arxiv.org/abs/2509.25149>. Version 2, revised 4 March 2026.
- [33] Open Compute Project. OCP microscaling formats (MX) specification, version 1.0. Technical report, Open Compute Project, 2023. URL <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf>.

- [34] Kaan Ozkara, Tao Yu, and Youngsuk Park. Stochastic rounding for LLM training: Theory and practice. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics*, volume 258 of *Proceedings of Machine Learning Research*, pages 4402–4410, 2025. URL <https://proceedings.mlr.press/v258/ozkara25b.html>.
- [35] Andrei Panferov, Erik Schultheis, Soroush Tabesh, and Dan Alistarh. Quartet II: Accurate LLM pre-training in NVFP4 by improved unbiased gradient estimation. *arXiv preprint arXiv:2601.22813*, 2026. doi: 10.48550/arXiv.2601.22813. URL <https://arxiv.org/abs/2601.22813>.
- [36] Mehdi Rahimifar, Amin Darabi, Mehran Taghian Jazi, Xing Huang, Yao Wang, Zhijun Tu, Yufei Cui, Yunke Peng, and Hongliang Li. Stable FP4 training via transposition-invariant block quantization. *arXiv preprint arXiv:2607.24953*, 2026. doi: 10.48550/arXiv.2607.24953. URL <https://arxiv.org/abs/2607.24953>.
- [37] Bitu Rouhani, Ritchie Zhao, Venmugil Elango, Rasoul Shafipour, Mathew Hall, Maral Mesmakhosroshahi, et al. With shared microexponents, a little shifting goes a long way. *arXiv preprint arXiv:2302.08007*, 2023. doi: 10.48550/arXiv.2302.08007. URL <https://arxiv.org/abs/2302.08007>.
- [38] Bitu Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023. doi: 10.48550/arXiv.2310.10537. URL <https://arxiv.org/abs/2310.10537>.
- [39] Benjamin F. Spector, Simran Arora, Aaryan Singhal, Daniel Y. Fu, and Christopher Re. ThunderKittens: Simple, fast, and adorable AI kernels. *arXiv preprint arXiv:2410.20399*, 2024. URL <https://arxiv.org/abs/2410.20399>.
- [40] Xiao Sun, Naigang Wang, Chia-Yu Chen, Jiamin Ni, Ankur Agrawal, Xiaodong Cui, Swagath Venkataramani, Kaoutar El Maghraoui, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. Ultra-low precision 4-bit training of deep neural networks. In *Advances in Neural Information Processing Systems*, volume 33, pages 1796–1807, 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/13b919438259814cd5be8cb45877d577-Paper.pdf.
- [41] Xingwu Sun, Shuaipeng Li, Ruobing Xie, Weidong Han, Kan Wu, Zhen Yang, et al. Scaling laws for floating-point quantization training. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57544–57570, 2025. URL <https://proceedings.mlr.press/v267/sun25j.html>.
- [42] Mehran Taghian, Yunke Peng, Xing Huang, Yao Wang, Yaoyuan Wang, Wei Guo, et al. HiFloat4 format for language model pre-training on Ascend NPUs. *arXiv preprint arXiv:2604.08826*, 2026. doi: 10.48550/arXiv.2604.08826. URL <https://arxiv.org/abs/2604.08826>.
- [43] Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, et al. 2 OLMo 2 furious. *arXiv preprint arXiv:2501.00656*, 2025. URL <https://arxiv.org/abs/2501.00656>.
- [44] Albert Tseng, Tao Yu, and Youngsuk Park. Training LLMs with MXFP4. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics*, volume 258 of *Proceedings of Machine Learning Research*, pages 1630–1638. PMLR, 2025. URL <https://proceedings.mlr.press/v258/tseng25a.html>.

- [45] Ruizhe Wang, Yeyun Gong, Xiao Liu, Guoshuai Zhao, Ziyue Yang, Baining Guo, Zheng-Jun Zha, and Peng Cheng. Optimizing large language model training using FP4 quantization. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 62937–62957, 2025. URL <https://proceedings.mlr.press/v267/wang25ae.html>.
- [46] Erik Wijmans, Brody Huval, Alexander Hertzberg, Vladlen Koltun, and Philipp Krahenbuhl. Cut your losses in large-vocabulary language models. *arXiv preprint arXiv:2411.09009*, 2024. URL <https://arxiv.org/abs/2411.09009>.
- [47] Hanmei Yang, Summer Deng, Amit Nagpal, Maxim Naumov, Mohammad Janani, Tongping Liu, and Hui Guan. An empirical study of microscaling formats for low-precision LLM training. In *IEEE Symposium on Computer Arithmetic*, 2025. doi: 10.1109/ARITH64983.2025.00011. URL <https://doi.org/10.1109/ARITH64983.2025.00011>.
- [48] Wuyue Zhang, Chongdong Huang, Chunbo You, Cheng Gu, Fengjuan Wang, and Mou Sun. Practical FP4 training for large-scale MoE models on hopper GPUs. *arXiv preprint arXiv:2603.02731*, 2026. URL <https://arxiv.org/abs/2603.02731>.
- [49] Qian Zhao, Kunlong Chen, Changxin Tian, Zhonghui Jiang, Haitao Zhang, Chaofan Yu, Peijie Jiang, Mingliang Gong, Jia Liu, Ziqi Liu, Zhiqiang Zhang, and Jun Zhou. Rethinking shrinkage bias in LLM FP4 pretraining: Geometric origin, systemic impact, and UFP4 recipe. *arXiv preprint arXiv:2606.20381*, 2026. URL <https://arxiv.org/abs/2606.20381>.
- [50] Jiecheng Zhou, Ding Tang, Rong Fu, Boni Hu, Haoran Xu, Yi Wang, Zhilin Pei, Zhongling Su, Liang Liu, Xingcheng Zhang, and Weiming Zhang. Towards efficient pre-training: Exploring FP4 precision in large language models. *arXiv preprint arXiv:2502.11458*, 2025. doi: 10.48550/arXiv.2502.11458. URL <https://arxiv.org/abs/2502.11458>.

A Implementation and Validation Notes

The kernels specialize the production tensor shapes and define buffer lifetimes across eager execution, recomputation, and CUDA Graph replay. QKV and FFN producers emit both quantized orientations. Residual-RMSNorm preserves the BF16 residual and exact row statistics required by backward. Each path is checked against a higher-precision reference for outputs, input gradients, weight gradients, relative RMS error, and zero fractions before model timing.

Row- and column-oriented scales are distinct numerical state, not interchangeable layout metadata. Tensors returned to autograd cannot alias a workspace whose lifetime ends with the producing stream. Each delayed FP4 prefetch buffer has one owner and is released exactly once, preventing asynchronous work from outliving its storage. Distributed measurements use per-GPU timing and dispatch evidence because the slowest GPU sets the step time.

A.1 Weight orientation and the fixed sign pattern

The custom global NVFP4 path creates one 2D16 numerical weight encoding and exposes it in both consumer layouts. The native 2D32 MXFP4 path follows the same orientation-consistency rule.

The signed-Hadamard paths use the fixed literal 0x00002817. Under the custom kernel’s convention, a set bit denotes +1 and a clear bit denotes −1. Bits 0–15 encode the same 16-entry sign vector that the pinned TE implementation stores explicitly. The H32 implementation applies this same 16-sign pattern separately to two 16-value halves, then combines the halves with the final 32-value Hadamard add/subtract stage. Thus H32 names the 32-value transform width, not a 32-bit random mask: the reported H32 transform repeats one signed-H16 diagonal and does not interpret the literal’s upper zero bits as 16 additional signs. No signs are drawn during training. The construction remains orthogonal and preserves the unquantized Wgrad dot product because the identical transform is paired on X and dY . The experiments do not compare this fixed vector with newly sampled signs and do not establish that the particular diagonal is optimal.

Compilation and first-step kernel generation are excluded from steady-state probe windows. Each measurement records device count, local batch, gradient accumulation, seed, loss implementation, source revision, and metric window. Each distributed worker checkpoints its stochastic-rounding seed and counter offset so that a resumed run does not reuse random-number draws.

Table 9 gives secondary production measurements. All three use a BF16 output projection and compiled regular CE. Their batch geometries and measurement windows differ, so the table does not isolate one implementation change.

Table 9: Compiled-CE B300 production snapshots. BF16 MFU uses the B300’s 2,250 TFLOP/s/GPU dense BF16 peak [30].

Route	LB/GA	Tokens/s/GPU	BF16 MFU	HBM/GPU
MXFP4 + row-SR	4/4	39,306.59	101.17%	156.91–157.15 GiB
CTA-local NVFP4	8/2	36,728.43	94.54%	211.05–211.16 GiB
27/5 depth hybrid	8/2	37,394.00	96.25%	228.28–233.42 GiB

B Low-Precision Output Head: Negative Result

This appendix records an engineering screen for which the exact low-precision forward format was not preserved. The vocabulary projection has a large forward GEMM and is therefore an attractive FP4 target. In the complete trainer, however, the head also owns cross entropy, the hidden-state gradient, the weight gradient, quantization, and saved-tensor lifetime. Accelerating one GEMM did not accelerate that whole boundary.

MXFP8 below denotes an eight-bit microscaling format used for one weight-gradient configuration.

Table 10: Historical whole-trainer output-head screen. Deltas are paired within one experiment. The complete profiler metadata was not preserved, so rates are rounded and are not compared with the primary performance panel.

Forward / hidden-state gradient / weight gradient	Tokens/s/GPU	Difference	Outcome
BF16 / BF16 / BF16	~34.00K	baseline	BF16 control
lowp [†] / BF16 / BF16	~33.95K	~ −0.16%	Loss differed from control
lowp [†] / BF16 / MXFP8	~33.91K	~ −0.27%	Loss and gradients differed

[†] The experiment record preserves the backward choices and paired rates but not the exact low-precision forward format. We therefore retain the literal “lowp” label rather than assigning a more specific format after the fact.

Both selective low-precision variants are slower than their matched BF16 control. The preserved record also says that neither reproduced the BF16 control loss, and that the MXFP8 weight-gradient configuration additionally changed gradients; it does not retain a publication-ready error magnitude, so we do not rank the size of those numerical differences. These results support the selected design: a BF16 output projection with compiled CE.

C Reproducibility

C.1 Report inputs

The main training and performance tables and figures are generated from checked-in data snapshots. The publication data directory contains the training-loss series, same-accelerator performance aggregates, checkpoint inventories, evaluation results, plotting programs, and a SHA-256 ledger. Figure generators verify their declared inputs before writing an output. The machine-readable recipe ledger records the route, precision policy, rounding and preconditioning choices, batch geometry, seed when recovered, checkpoint horizon, and evidence status for every experiment discussed in the paper. Shared model, data, and optimizer fields are stored separately so that an incomplete historical row cannot silently inherit a value from a newer run.

For display consistency, the historical BF16 and TE-native training-loss curves in Figure 7 use fixed offsets inferred from 41 matched steps in seed-42 controls. The raw series and adjustment remain in the publication data.

The long-run curves combine only explicitly identified execution segments. The CTA-local NVFP4+fixed-H16 and operand-hybrid trajectories each cross a documented full-state resume boundary. Their manifests identify the source and continuation ranges; no overlapping step is counted twice. Training-loss observations for custom global NVFP4 are regular through step 19,000. The configured Weights & Biases run is no longer returned by the service, and the dense local CSV named by the historical manifest was not present in any recovered or released tree. Seventy-four exact observations survive after step 19,000 in fragmented training logs. The largest missing sequence of expected ten-step records runs from step 24,710 through step 33,000. The dashed late line is a visual LOWESS fit through the surviving observations, evaluated at the BF16 plotting steps. It is never used to compute a reported numeric endpoint, table entry, or statistic.

C.2 Checkpoint and evaluator contract

Training comparisons use exact shared steps. Custom-route values are raw observations. Downstream comparisons use literal step-38,000 checkpoints; a terminal, nearby, or checkpoint from a different training run is never substituted. Distributed checkpoints must contain all expected shards and pass conversion checks before evaluation.

The evaluator must reproduce the training model’s RoPE and RMSNorm semantics. The Llama-3.1 8B training configuration uses base 500,000 and the long-context RoPE scaling parameters (8, 1, 4, 8192). An earlier evaluator disabled this scaling in both the converted-checkpoint evaluator and native TorchTitan reference. Agreement between those two implementations therefore did not

establish agreement with the training model. Results from that evaluator are excluded.

An earlier validation stream used the correct model semantics but failed a data-source robustness check: apparent route ordering changed when examples came from different contiguous regions of the source datasets. Those values are also excluded. The replacement stream was constructed afresh from deterministic windows in the 90–95% physical sample range of the DCLM and OLMo-without-DCLM components. Documents are deduplicated by raw-text SHA-256, combined at an 82/18 token ratio, globally hash-shuffled, and only then packed and sharded. Thus no GPU worker is restricted to one source or contiguous region of source data.

The replacement fixed validation stream contains 768 sequences of 8,193 tokens, yielding 6,291,456 scored next-token targets. It is tokenized once with the pinned Llama-3.1 tokenizer and reused byte-for-byte for every checkpoint. We call it a *fixed external validation stream*, not a held-out split, because zero document overlap across every historical training trajectory has not been proven. Its source ledger contains 4,597 documents with 4,597 unique raw-text hashes. Before reporting route comparisons, the corrected checkpoint evaluator must agree exactly with the native TorchTitan model at the full 8,192-token context, and a late BF16 checkpoint must improve over an early BF16 checkpoint by more than five sequence-level standard errors. Both checks pass. The converted checkpoint and native TorchTitan model agree exactly at full context for the step-2,000 and step-29,000 BF16 checkpoints. Their validation NLL values are 3.283272 and 2.460520, respectively; the paired late-minus-early difference is -0.822753 with sequence-level standard error 0.003708.

Downstream evaluation uses BF16 inference and one evaluator configuration for all routes. MMLU is 5-shot, HellaSwag 10-shot, WinoGrande 5-shot, and ARC-Challenge 25-shot; HellaSwag and ARC report normalized accuracy. A route enters the common panel only after the exact checkpoint and corrected model semantics are verified.

C.3 Historical execution identity

Historical experiments are bound to the code, submodule, kernel asset, checkpoint, and output identities recovered for that execution. The current source and runtime tips are not assigned retroactively to an older run. Where a field was not recovered, the ledger records it as unknown rather than guessing the nearest commit. Complete identifiers and content digests are in the publication data ledger rather than the narrative.

The custom global NVFP4 path uses one orientation-consistent 2D16 weight encoding. The code used by the completed MXFP4+row-SR+fixed-H32 run is content-verified in its run manifest. The CTA-local NVFP4+fixed-H16 and operand-wise plain-H16 trajectories each join verified execution segments at an exact full-state checkpoint. Their manifests record implementation changes, segment boundaries, and checkpoint state.

C.4 Controlled Dgrad diagnostic

The operand-format diagnostic uses tensors from the step-28,000 checkpoint of the completed MXFP4+row-SR+fixed-H32 trajectory. It computes Dgrad with MXFP4 and CTA-local NVFP4 against the corresponding BF16 result. The diagnostic configures a paired plain-H16 transform for Wgrad only; that transform does not enter the scored Dgrad contraction and is not part of the comparison.

For each captured tensor, four stochastic candidate tensors are averaged before computing

$$E = \sum_i \left(\widehat{dX}_i - dX_i^{\text{BF16}} \right)^2. \quad (15)$$

The capture covers three sequence-2,048 batches, QKV, attention-output, and FFN projections in all 32 blocks, and an independently seeded stochastic-rounding generator rather than the training run’s saved generator state. This gives 96 layer–component comparisons per batch and 288 in total. CTA-local NVFP4 has lower error energy in all 288 comparisons. Giving each tensor equal weight, the cross-capture geometric mean of $E_{\text{MX}}/E_{\text{localCTA}}$ is 1.292.

C.5 Excluded experiments

The custom global NVFP4/MXFP4 depth hybrid remains outside the common endpoint panels because it diverged and retained only 16 of 32 terminal shards. The TE route with four final BF16 blocks and the fixed-H32 operand hybrid both completed 160B-token trajectories with resumable final checkpoints and are included in the endpoint panels.

Diagnostic screens are also excluded from the common panels. The low-precision output-head configurations produced no stable model-level speedup. A CTA-local NVFP4 transform screen resumed near the end of training under a changed runtime, so it is not a causal comparison. The superseded downstream evaluator omitted training-time RoPE scaling, while the superseded validation stream was sensitive to the contiguous source-data region selected. Their records are retained for audit, not used as scientific endpoints.

Run `make` in the report directory to regenerate the figures and PDF. The checked-in data README lists each source file, generator, and verification command; `data/SHA256SUMS` binds the publication inputs.

C.6 Code and data availability

Code, pinned kernel sources, experiment recipes, evaluation tools, and figure builders are available at <https://github.com/MrHuff/mfu-fp4-training>. Project-authored software is released under the MIT License. The original manuscript and figures are released under CC BY 4.0; included third-party components retain their own terms. Training corpora, tokenizer assets, model checkpoints, and checkpoint metadata are not redistributed. The repository instead provides hash-bound input schemas and public recipes for binding authorized local copies without recording credentials or private storage locations.